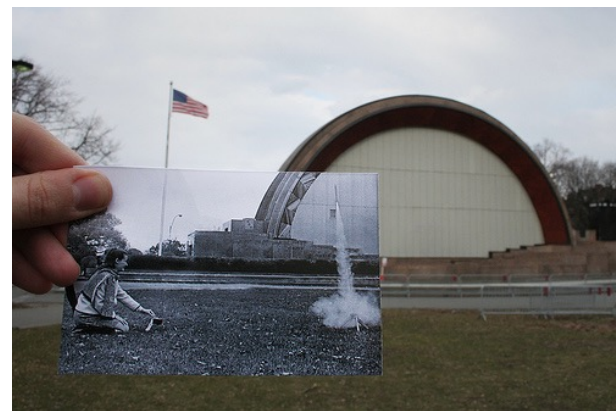


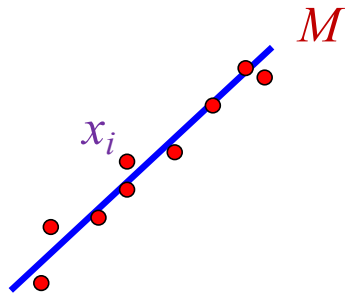
Image alignment



[Source](#)

Alignment: Fitting of transformations

- Previously: fitting a model to features in one image

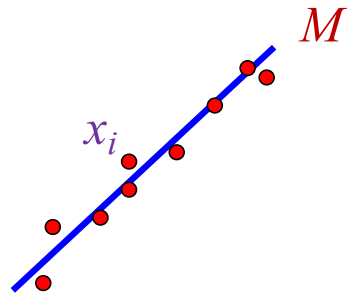


- Given: points $x_1, \dots, x_n$
- Find: model M that minimizes

$$\sum_i L(r(x_i, M))$$

Alignment: Fitting of transformations

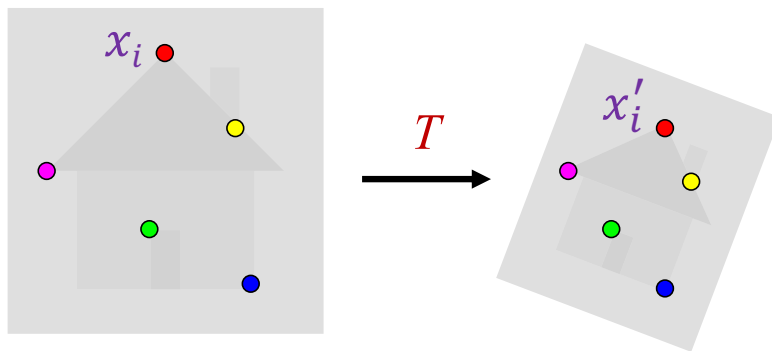
- Last time: fitting a model to features in one image



- Given: points $x_1, \dots, x_n$
- Find: model M that minimizes

$$\sum_i L(r(x_i, M))$$

- Alignment: fitting a model to a transformation between pairs of features (*matches*) in two images



- Given: matches $(x_1, x'_1), \dots, (x_n, x'_n)$
- Find: transformation T that minimizes

$$\sum_i L(r(T(x_i), x'_i))$$

Alignment: Overview

- Motivation
- Fitting of transformations
 - Affine transformations
 - Homographies
- Robust alignment
 - Descriptor-based feature matching
 - RANSAC
- Learning-based feature extraction and matching
- Large-scale alignment

Alignment applications: Panorama stitching



<http://matthewalunbrown.com/autostitch/autostitch.html>

Alignment applications: Instance recognition



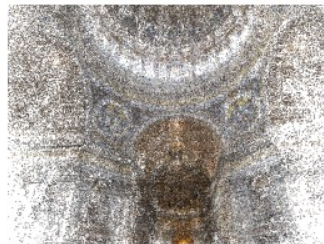
Alignment applications: Large-scale reconstruction



Colosseum: 2,097 images, 819,242 points



Trevi Fountain: 1,935 images, 1,055,153 points



Pantheon: 1,032 images, 530,076 points

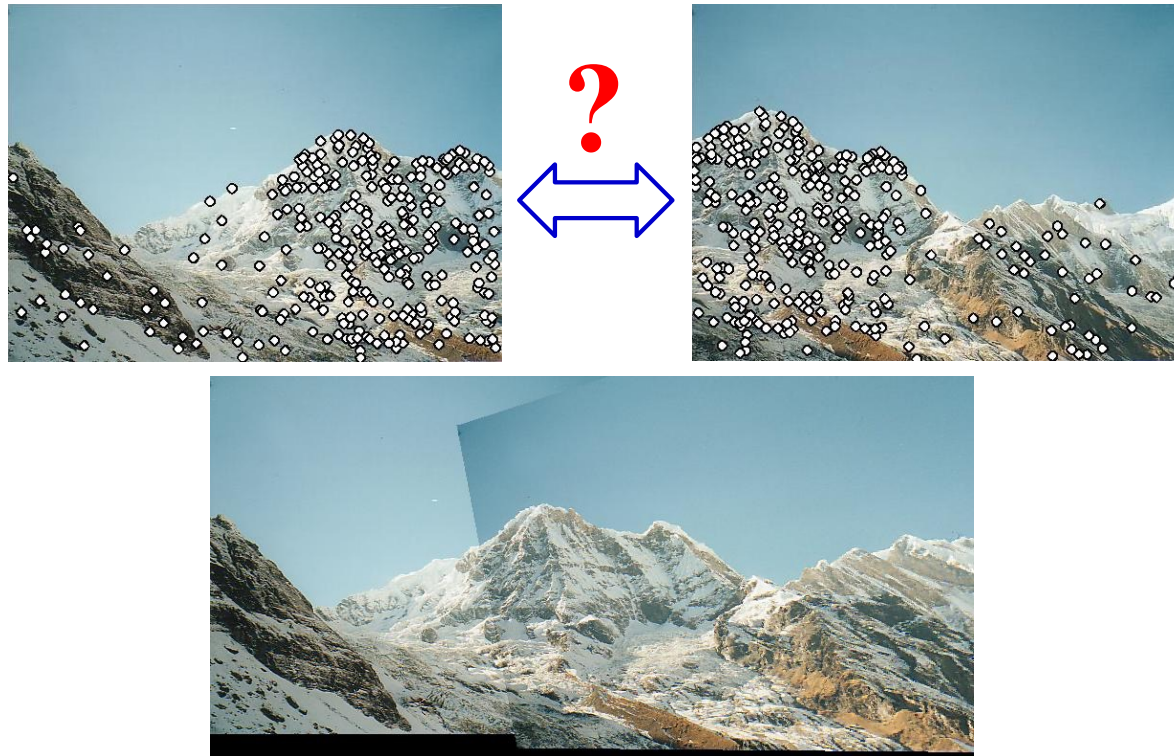


Hall of Maps: 275 images, 230,182 points

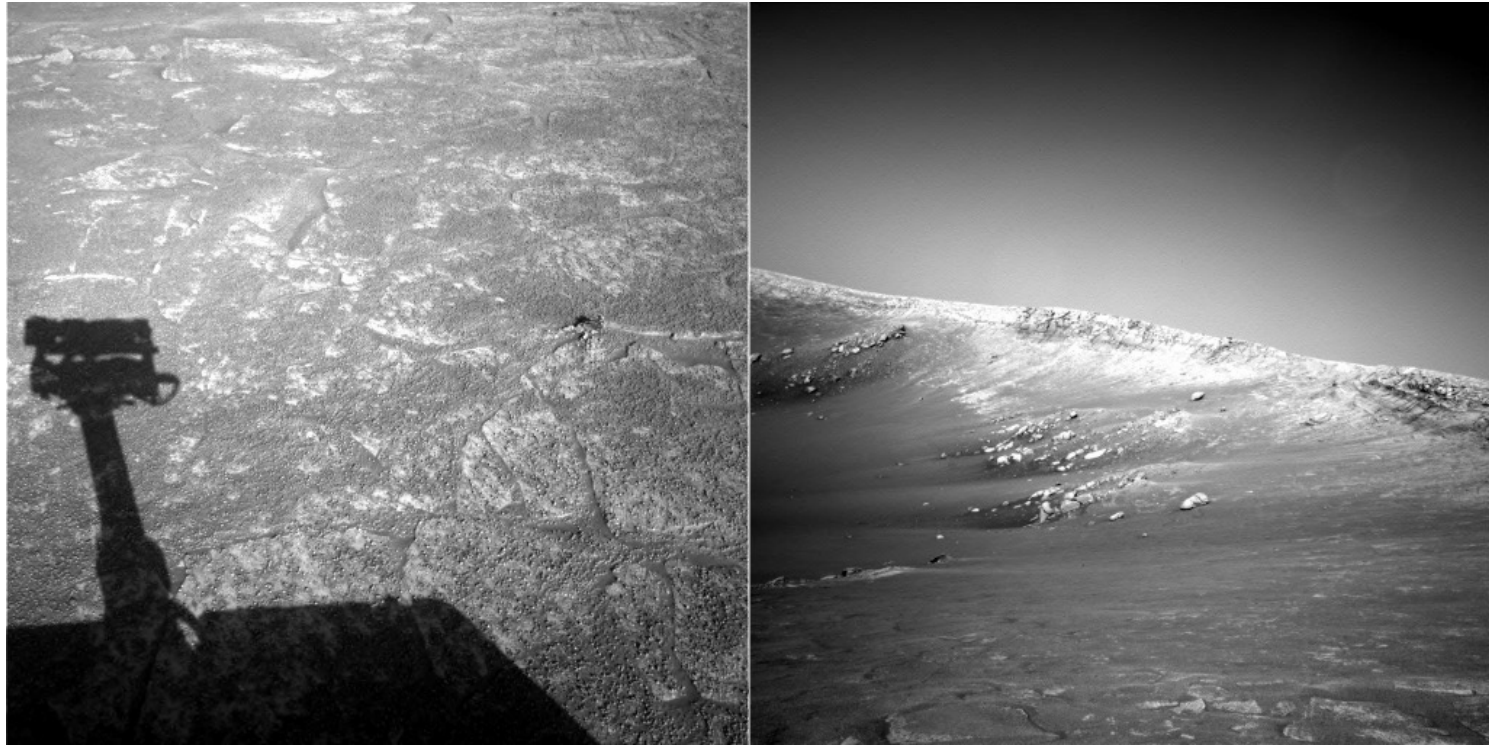
S. Agarwal et al. [Building Rome in a Day](#). ICCV 2009

Feature-based alignment

- Find a set of feature matches that agree in terms of:
 - Local appearance
 - Geometric configuration



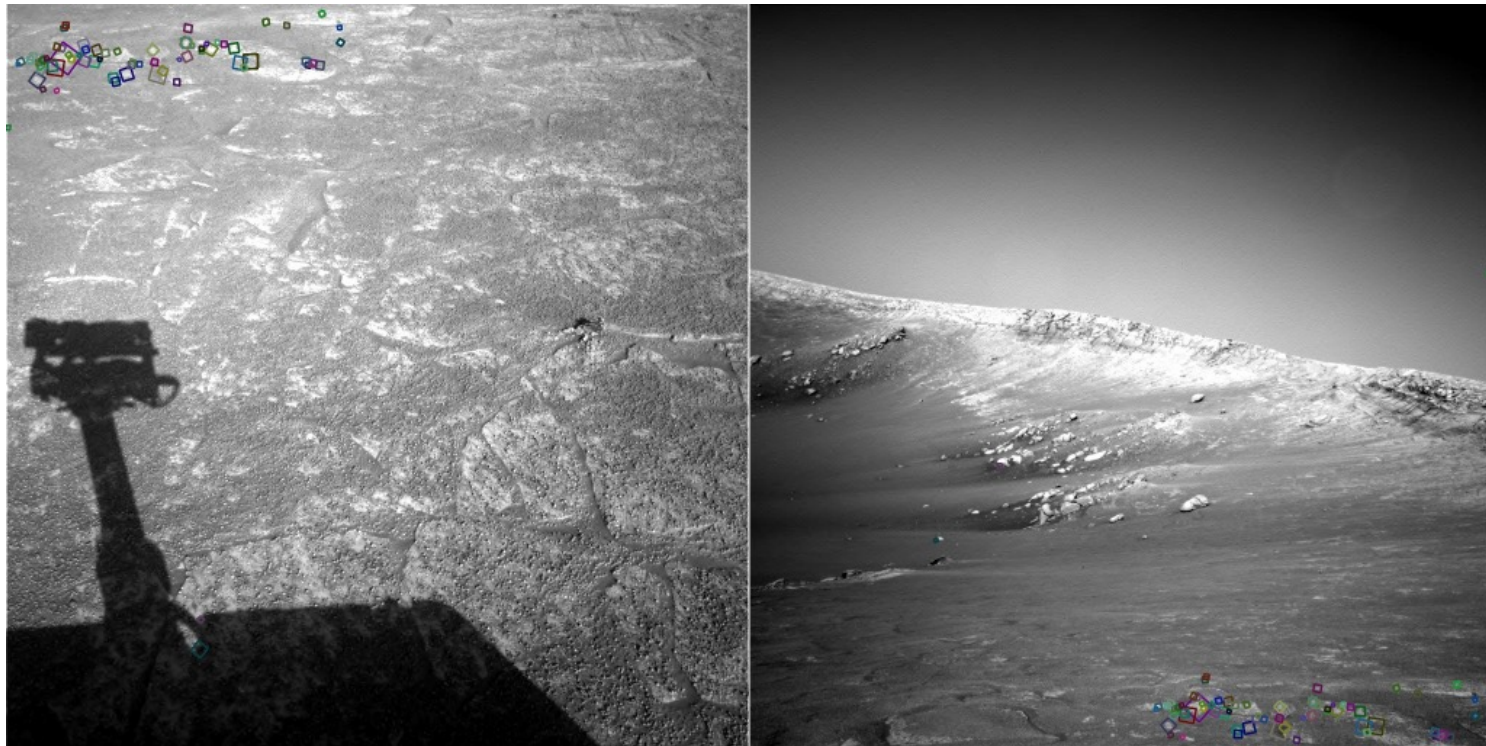
Feature-based alignment *really* works!



NASA Mars Rover images

Source: N. Snavely

Feature-based alignment *really works!*



**NASA Mars Rover images
with SIFT feature matches**

Source: N. Snavely

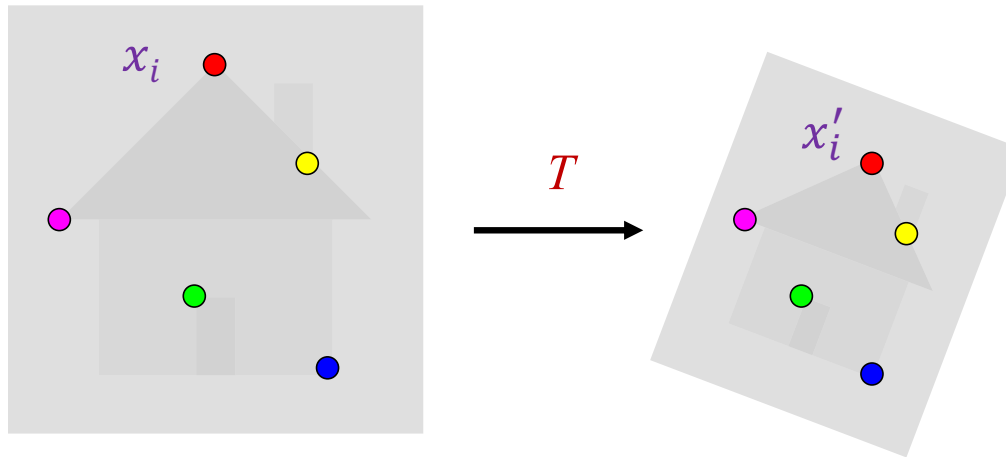
Alignment: Overview

- Motivation
- Fitting of transformations
 - Affine transformations
 - Homographies

Alignment: Fitting of transformations

- Given: matches $(x_1, x'_1), \dots, (x_n, x'_n)$
- Find: transformation T that minimizes

$$\sum_i L(r(T(x_i), x'_i))$$

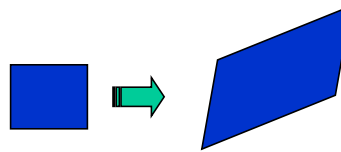


2D transformation models

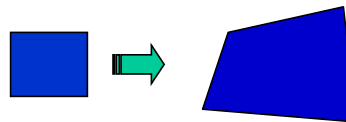
- Similarity
(translation,
scale, rotation)



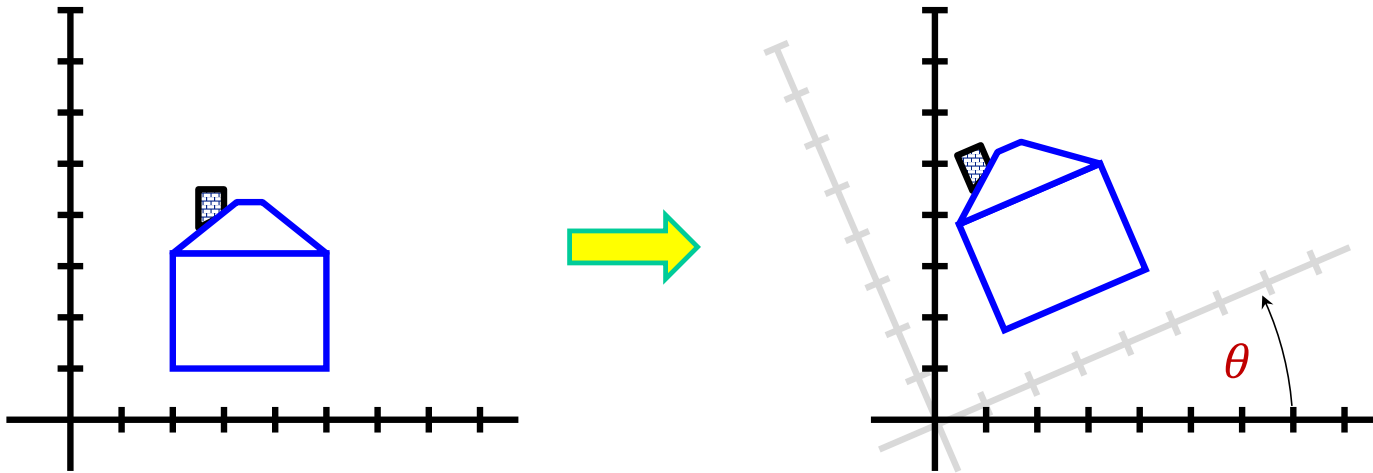
- Affine



- Projective
(homography)



Recall: Rotation

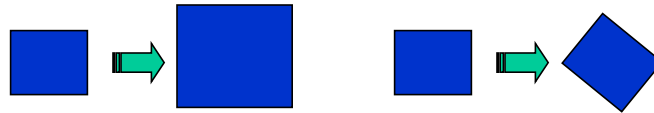


$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

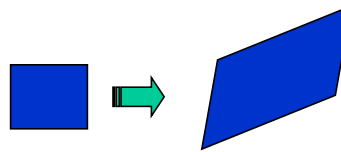
- Estimating the rotation matrix requires enforcing nonlinear constraints, so we will skip the details

2D transformation models

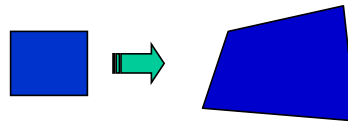
- Similarity
(translation,
scale, rotation)



- Affine

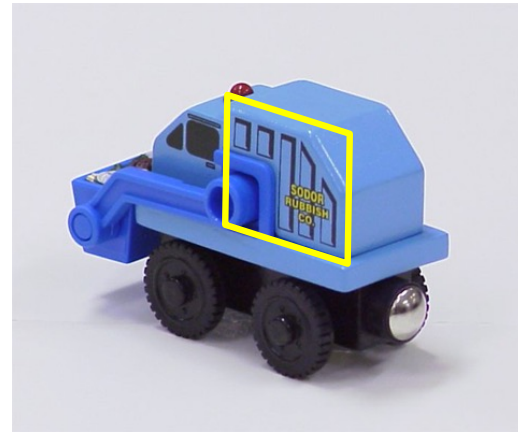


- Projective
(homography)



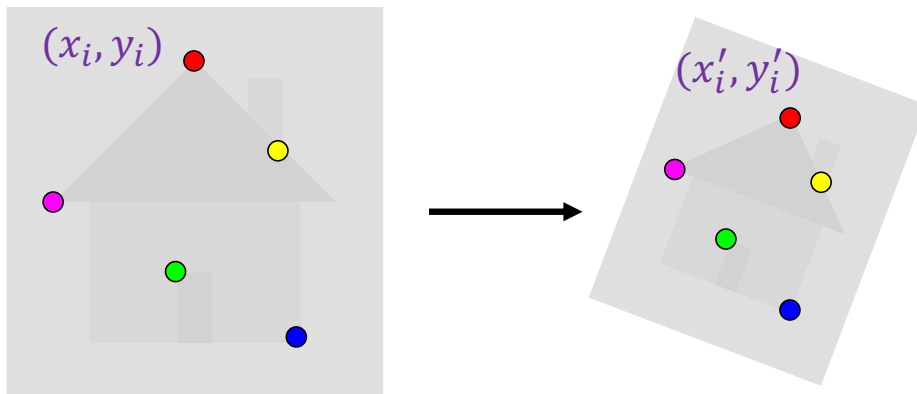
Let's start with affine transformations

- Simple fitting procedure: linear least squares
- Approximates viewpoint changes for roughly planar objects and roughly orthographic cameras
- Can be used to initialize fitting for more complex models



Fitting an affine transformation

- Assume we know the correspondences, how do we get the transformation?



$$\begin{pmatrix} x'_i \\ y'_i \end{pmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{pmatrix} x_i \\ y_i \end{pmatrix} + \begin{pmatrix} t_1 \\ t_2 \end{pmatrix}$$

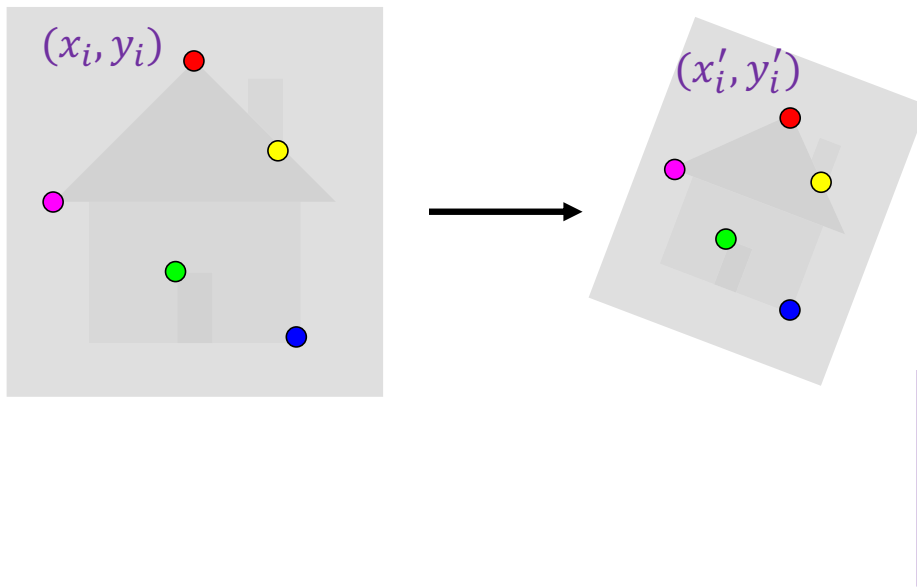
$x'_i \qquad \qquad \mathbf{M} \qquad \qquad x_i \qquad \qquad \mathbf{t}$

Want to find $\mathbf{M}$, $\mathbf{t}$ to minimize

$$\sum_{i=1}^n \|x'_i - \mathbf{M}x_i - \mathbf{t}\|^2$$

Fitting an affine transformation

- Assume we know the correspondences, how do we get the transformation?

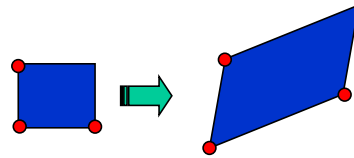


$$\begin{pmatrix} x'_i \\ y'_i \end{pmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{pmatrix} x_i \\ y_i \end{pmatrix} + \begin{pmatrix} t_1 \\ t_2 \end{pmatrix}$$

$$\begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{pmatrix} \dots \\ x'_i \\ y'_i \\ \dots \end{pmatrix}$$

Fitting an affine transformation

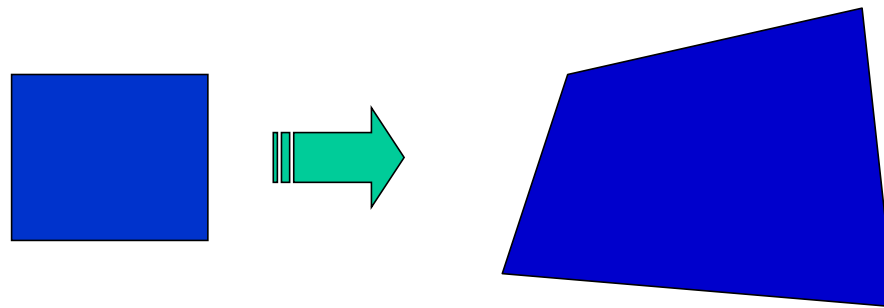
- How many matches do we need to solve for the transformation parameters?



$$\begin{bmatrix} x_i & y_i & \dots & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \\ & & \dots & & & \end{bmatrix} \begin{pmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{pmatrix} = \begin{pmatrix} \dots \\ x'_i \\ y'_i \\ \dots \end{pmatrix}$$

Fitting a homography

- A *homography* is a plane projective transformation (transformation taking a quad to another arbitrary quad)

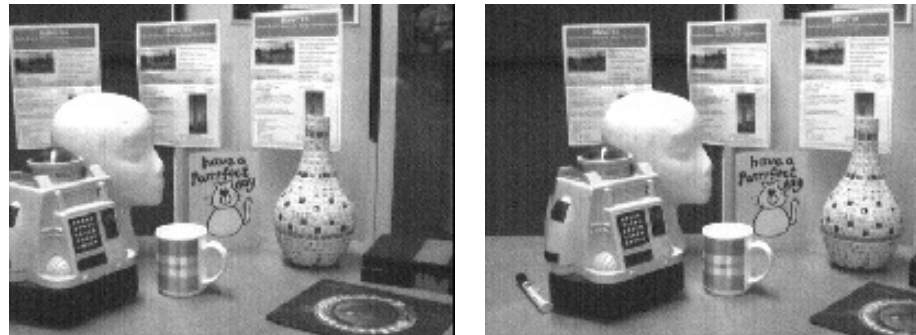


Homography in the real world

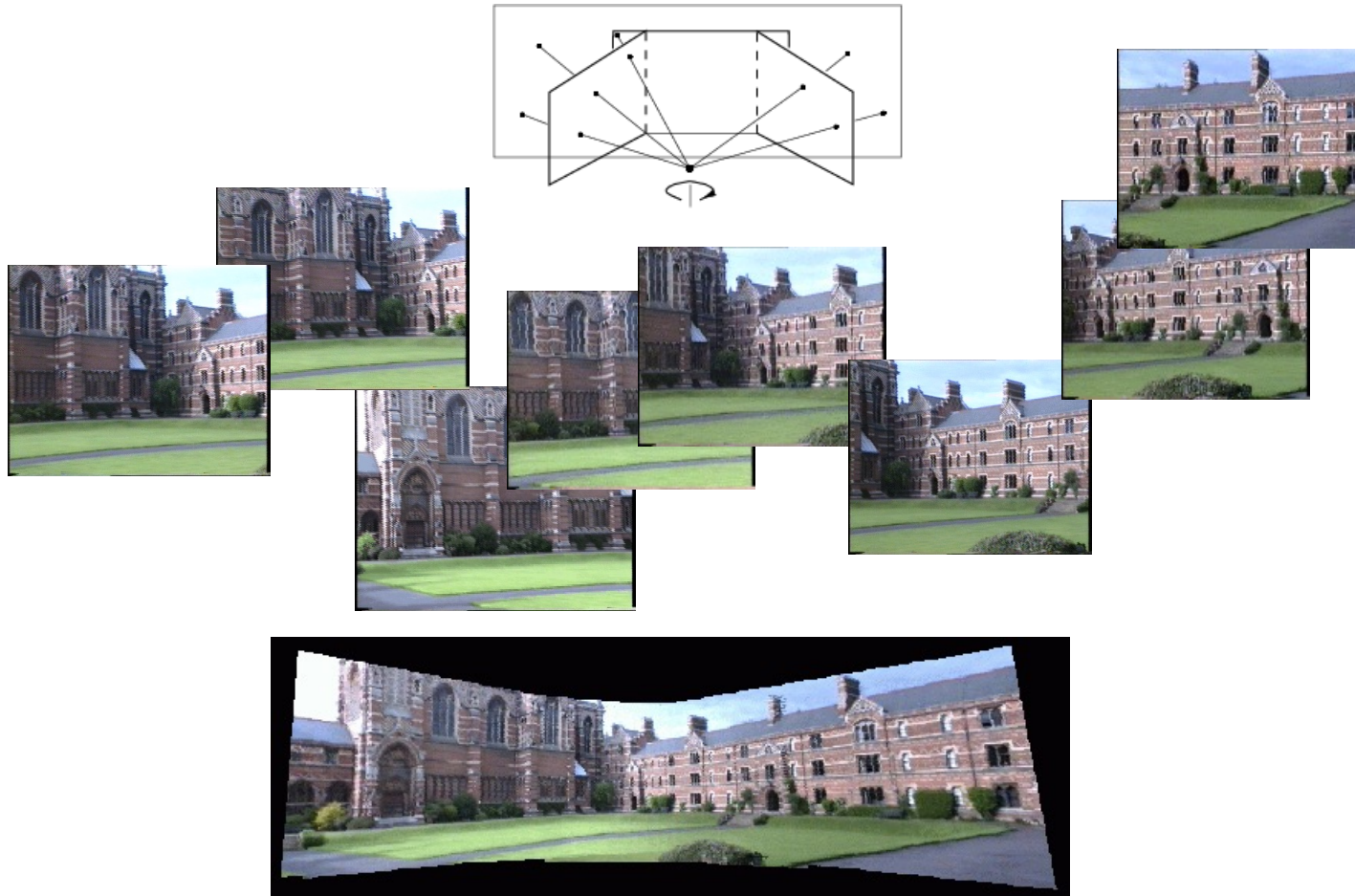
- The transformation between two views of a planar surface



- The transformation between images from two cameras that share the same center



Homography for panorama stitching



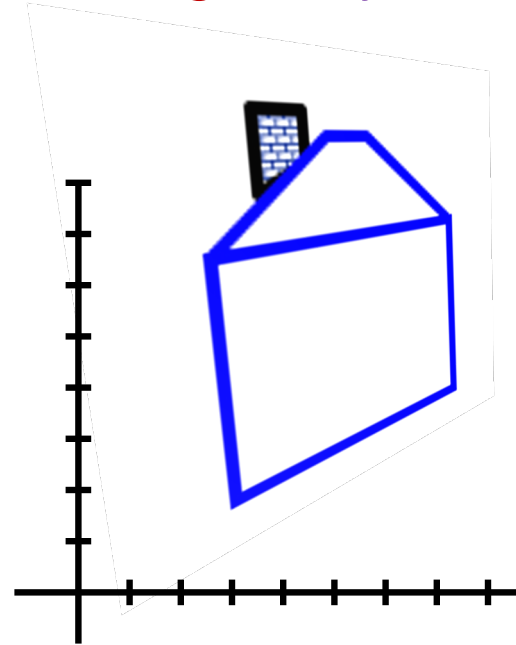
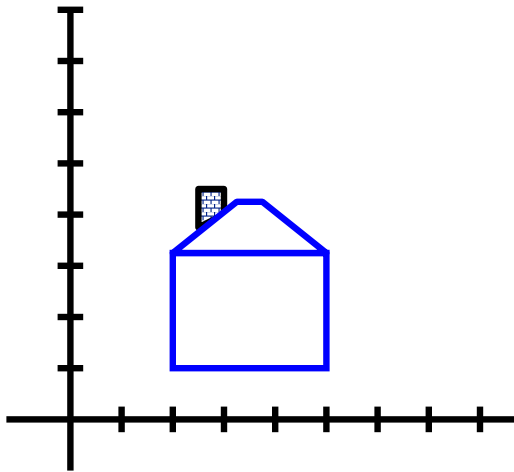
Source: Hartley & Zisserman

Fitting a homography

- Recall:

$$x' = \frac{ax + by + c}{gx + hy + i},$$

$$y' = \frac{dx + ey + f}{gx + hy + i}$$



Fitting a homography

- We need 2D *homogeneous coordinates*:

$$(x, y) \Rightarrow \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Converting *to* homogeneous
coordinates

$$\begin{pmatrix} x \\ y \\ w \end{pmatrix} \Rightarrow (x/w, y/w)$$

Converting *from* homogeneous
coordinates

- All homogeneous coordinate vectors that are scalar multiples of each other represent the same point!
- Equation for homography in homogeneous coordinates:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} \cong \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$\mathbf{x}' \cong \mathbf{H}\mathbf{x}$
“equal up to scale”

Fitting a homography

- Constraint from a match $(\mathbf{x}_i, \mathbf{x}'_i)$: $\mathbf{x}'_i \cong \mathbf{H}\mathbf{x}_i$
- How can we get rid of the scale ambiguity?
- Cross product trick: $\mathbf{x}'_i \times \mathbf{H}\mathbf{x}_i = \mathbf{0}$
- Recall cross product:

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} \times \begin{pmatrix} a' \\ b' \\ c' \end{pmatrix} = \begin{pmatrix} bc' - b'c \\ ca' - c'a \\ ab' - a'b \end{pmatrix}$$

- Let $\mathbf{h}_1^T, \mathbf{h}_2^T, \mathbf{h}_3^T$ be the rows of $\mathbf{H}$. Then

$$\mathbf{x}'_i \times \mathbf{H}\mathbf{x}_i = \begin{pmatrix} x'_i \\ y'_i \\ 1 \end{pmatrix} \times \begin{pmatrix} \mathbf{h}_1^T \mathbf{x}_i \\ \mathbf{h}_2^T \mathbf{x}_i \\ \mathbf{h}_3^T \mathbf{x}_i \end{pmatrix} = \begin{pmatrix} y'_i \mathbf{h}_3^T \mathbf{x}_i - \mathbf{h}_2^T \mathbf{x}_i \\ \mathbf{h}_1^T \mathbf{x}_i - x'_i \mathbf{h}_3^T \mathbf{x}_i \\ x'_i \mathbf{h}_2^T \mathbf{x}_i - y'_i \mathbf{h}_1^T \mathbf{x}_i \end{pmatrix}$$

Fitting a homography

- Constraint from a match $(\mathbf{x}_i, \mathbf{x}'_i)$:

$$\mathbf{x}'_i \times \mathbf{H} \mathbf{x}_i = \begin{pmatrix} x'_i \\ y'_i \\ 1 \end{pmatrix} \times \begin{pmatrix} \mathbf{h}_1^T \mathbf{x}_i \\ \mathbf{h}_2^T \mathbf{x}_i \\ \mathbf{h}_3^T \mathbf{x}_i \end{pmatrix} = \begin{pmatrix} y'_i \mathbf{h}_3^T \mathbf{x}_i - \mathbf{h}_2^T \mathbf{x}_i \\ \mathbf{h}_1^T \mathbf{x}_i - x'_i \mathbf{h}_3^T \mathbf{x}_i \\ x'_i \mathbf{h}_2^T \mathbf{x}_i - y'_i \mathbf{h}_1^T \mathbf{x}_i \end{pmatrix}$$

- Rearranging the terms:

$$\begin{bmatrix} \mathbf{0}^T & -x_i^T & y'_i x_i^T \\ x_i^T & \mathbf{0}^T & -x'_i x_i^T \\ -y'_i x_i^T & x'_i x_i^T & \mathbf{0}^T \end{bmatrix} \begin{pmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \mathbf{h}_3 \end{pmatrix} = \mathbf{0}$$

- Are these equations independent?

Fitting a homography

- Final linear system:

$$\begin{bmatrix} \mathbf{0}^T & \mathbf{x}_1^T & -y'_1 \mathbf{x}_1^T \\ \mathbf{x}_1^T & \mathbf{0}^T & -x'_1 \mathbf{x}_1^T \\ \dots & \dots & \dots \\ \mathbf{0}^T & \mathbf{x}_n^T & -y'_n \mathbf{x}_n^T \\ \mathbf{x}_n^T & \mathbf{0}^T & -x'_n \mathbf{x}_n^T \end{bmatrix} \begin{pmatrix} h_1 \\ h_2 \\ h_3 \end{pmatrix} = \mathbf{0} \quad A\mathbf{h} = \mathbf{0}$$

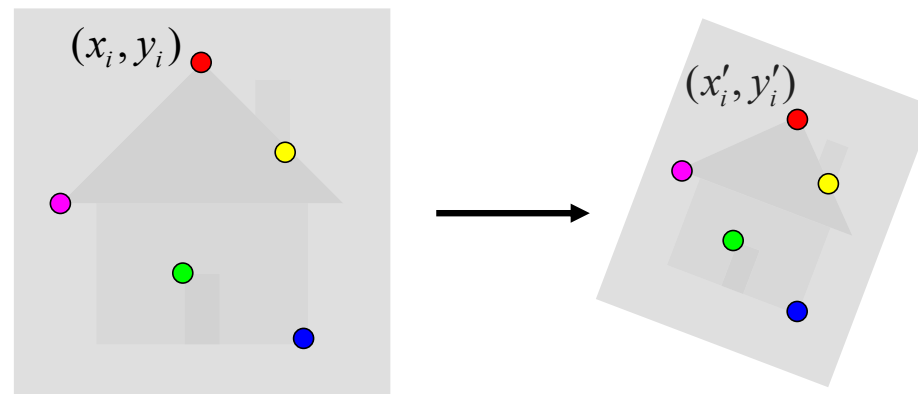
- Homogeneous least squares: find $\mathbf{h}$ minimizing $\|A\mathbf{h}\|^2$
 - Solution is eigenvector of $A^T A$ corresponding to smallest eigenvalue
- What is the minimum number of matches needed for a solution?
 - Four: A has 8 degrees of freedom (9 parameters, but scale is arbitrary), one match gives us two linearly independent equations

Alignment: Overview

- Motivation
- Fitting of transformations
 - Affine transformations
 - Homographies
- Robust alignment
 - Descriptor-based feature matching
 - RANSAC

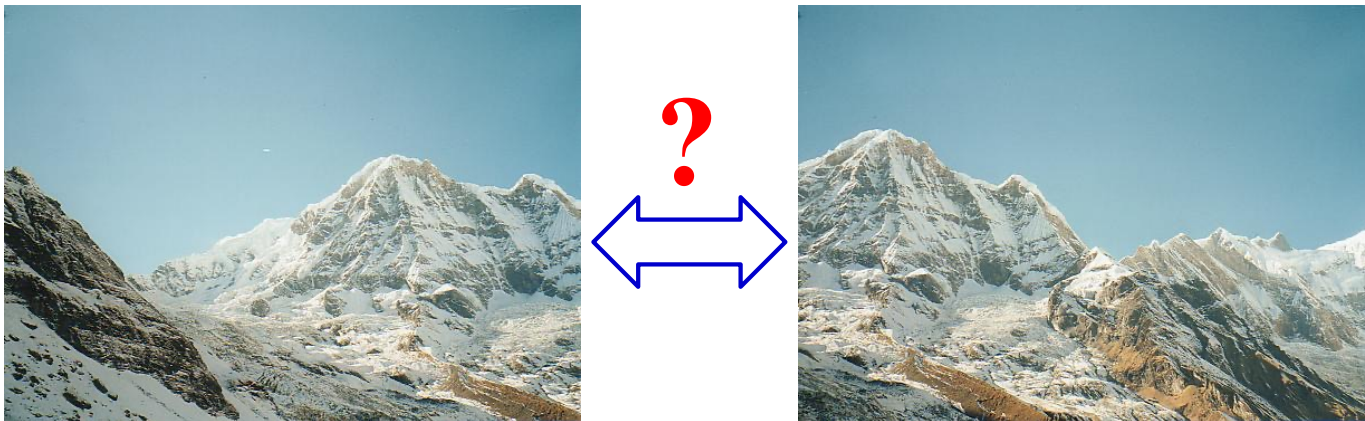
Robust feature-based alignment

- So far, we've assumed that we are given a set of correspondences between the two images we want to align
- What if we don't know the correspondences?



Robust feature-based alignment

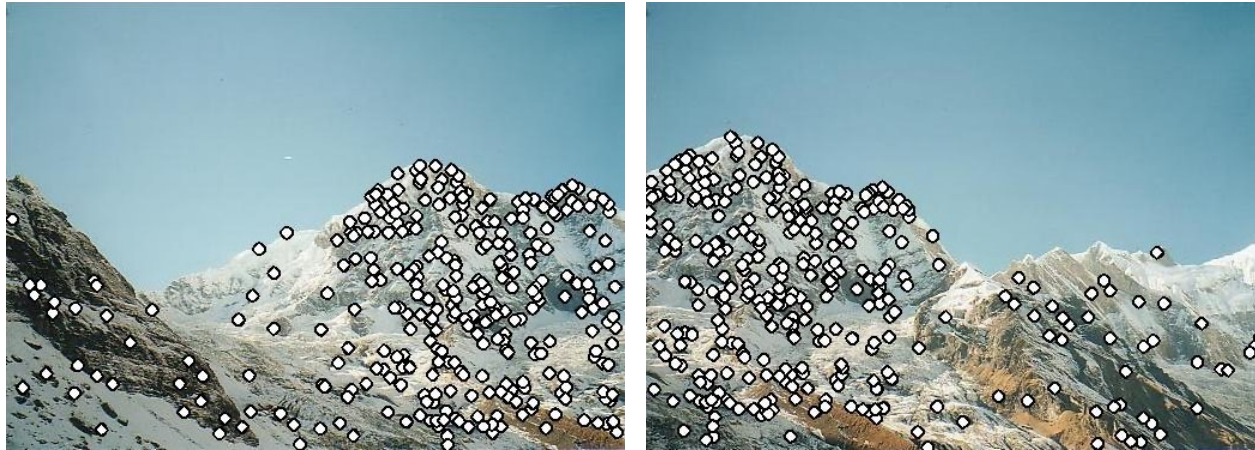
- So far, we've assumed that we are given a set of correspondences between the two images we want to align
- What if we don't know the correspondences?



Robust feature-based alignment: Overview



Robust feature-based alignment: Overview



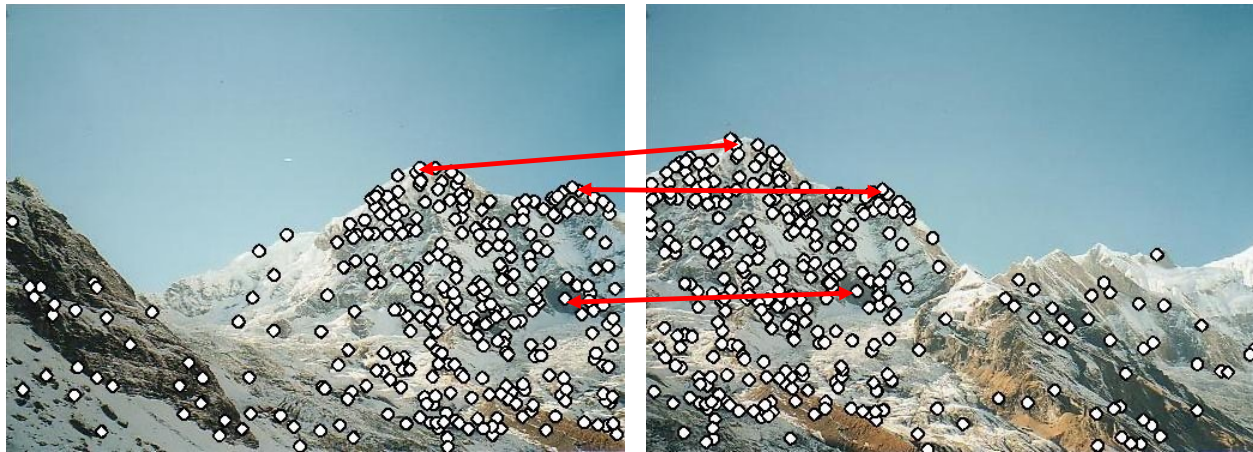
- Extract features

Robust feature-based alignment: Overview



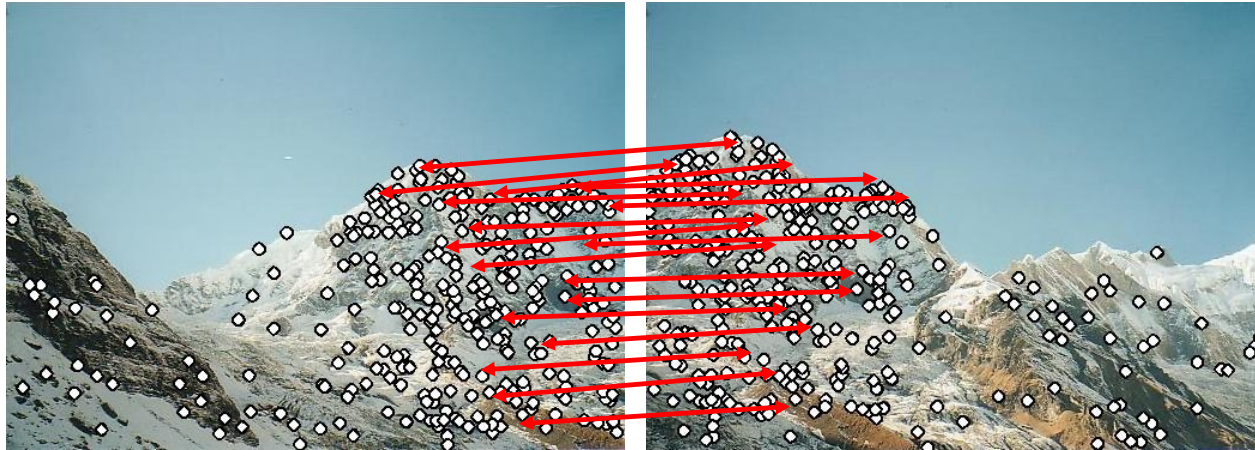
- Extract features
- Compute *putative matches*

Robust feature-based alignment: Overview



- Extract features
- Compute *putative matches*
- Loop:
 - Hypothesize transformation T

Robust feature-based alignment: Overview



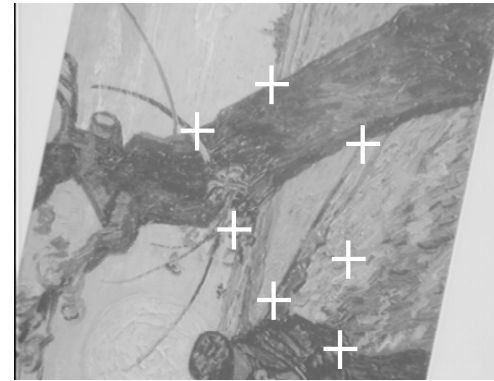
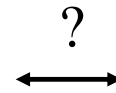
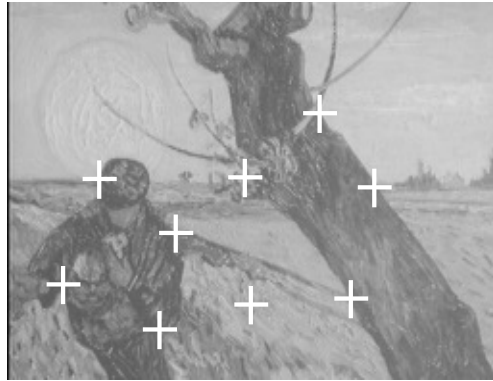
- Extract features
- Compute *putative matches*
- Loop:
 - *Hypothesize* transformation T
 - *Verify* transformation (search for other matches consistent with T)

Robust feature-based alignment: Overview

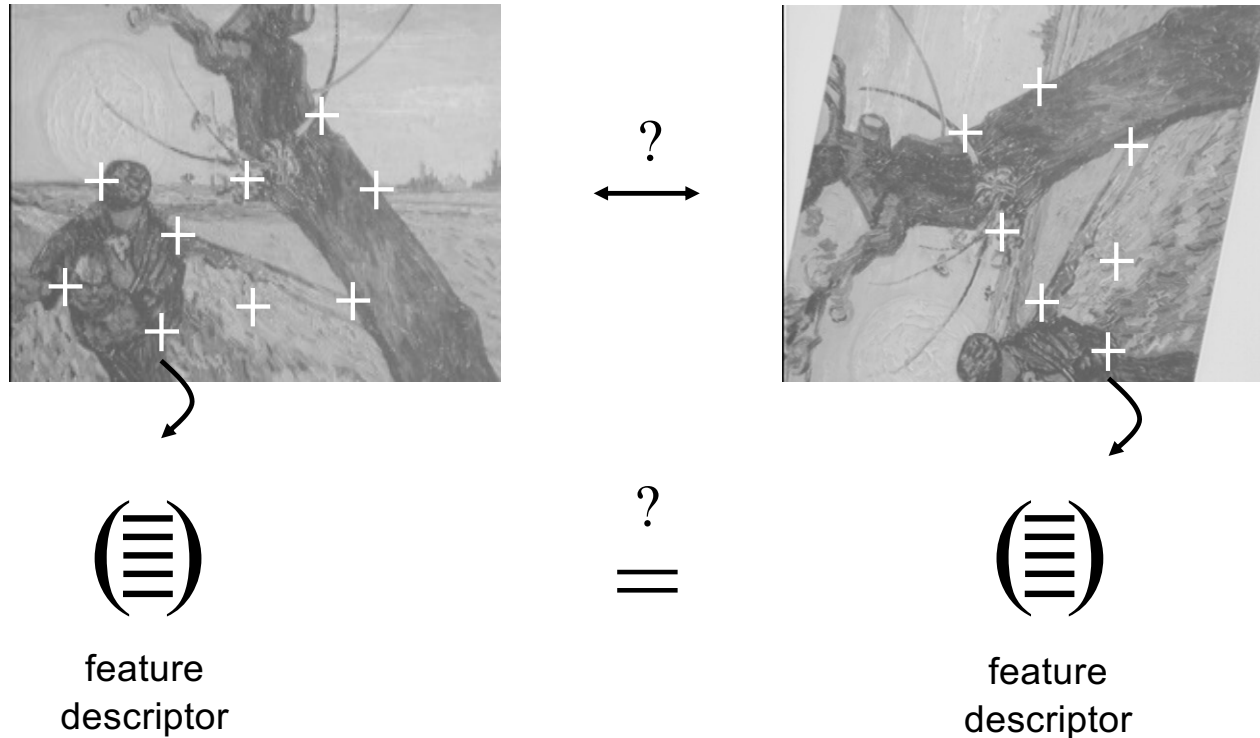


- Extract features
- Compute *putative matches*
- Loop:
 - *Hypothesize* transformation T
 - *Verify* transformation (search for other matches consistent with T)

Generating putative correspondences



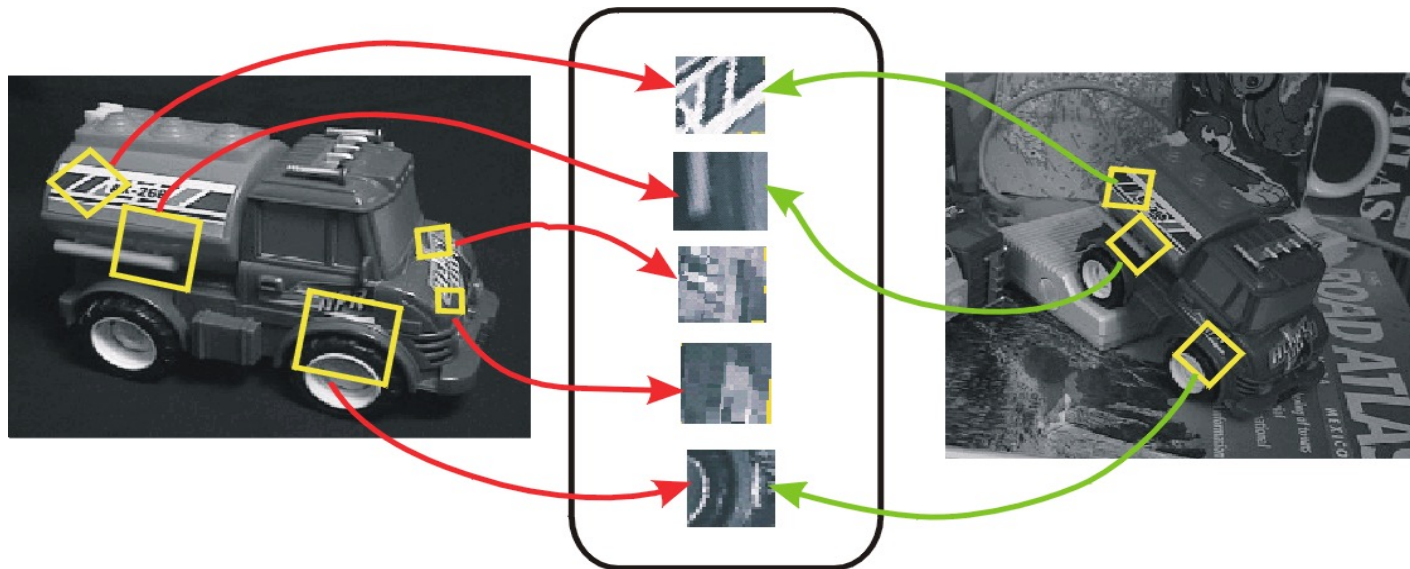
Generating putative correspondences



- Need to compare *feature descriptors* of local patches surrounding interest points

Feature descriptors

- Recall: feature detection vs. feature description



Comparing feature descriptors

- Simplest descriptor: vector of raw intensity values
- How to compare two such vectors $\mathbf{u}$ and $\mathbf{v}$?
 - **Sum of squared differences (SSD):**

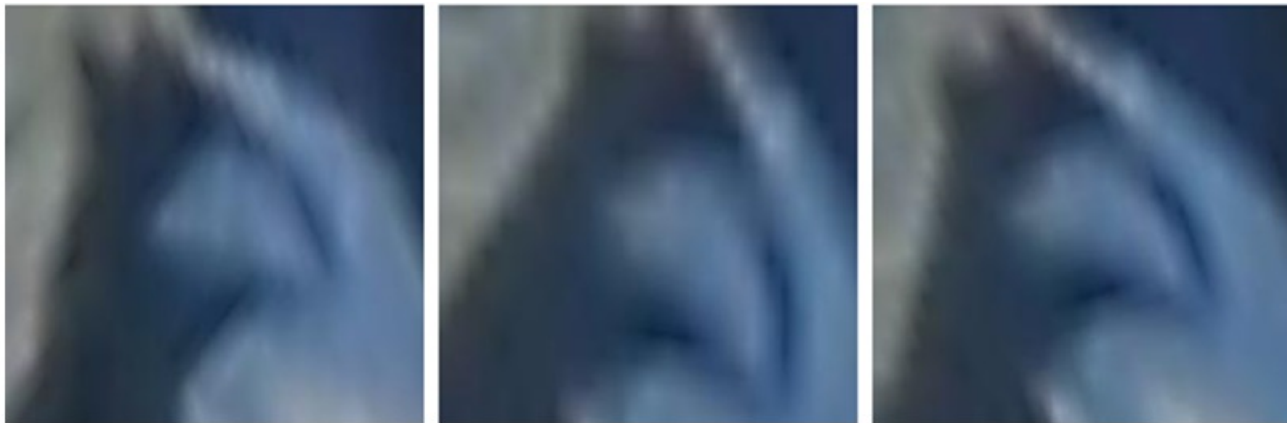
$$\text{SSD}(\mathbf{u}, \mathbf{v}) = \sum_i (u_i - v_i)^2$$

- **Normalized correlation:** dot product between $\mathbf{u}$ and $\mathbf{v}$ normalized to have zero mean and unit norm:

$$\rho(\mathbf{u}, \mathbf{v}) = \frac{\sum_i (u_i - \bar{u})(v_i - \bar{v})}{\sqrt{(\sum_j (u_j - \bar{u})^2)(\sum_j (v_j - \bar{v})^2)}}$$

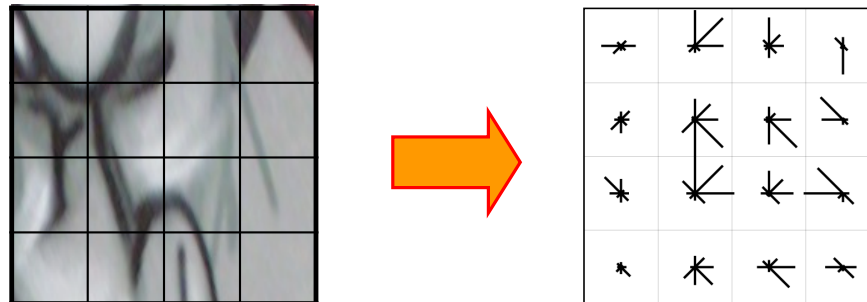
Comparing feature descriptors

- Small deformations can affect pixel-based metrics (SSD, cross-correlation) a lot



Feature descriptors: SIFT

- Descriptor computation:
 - Divide patch into 4×4 sub-patches
 - Compute histogram of gradient orientations (8 reference angles) inside each sub-patch
 - Resulting descriptor: $4 \times 4 \times 8 = 128$ dimensions



D. Lowe. [Distinctive image features from scale-invariant keypoints](#). *IJCV* 60 (2), pp. 91-110, 2004

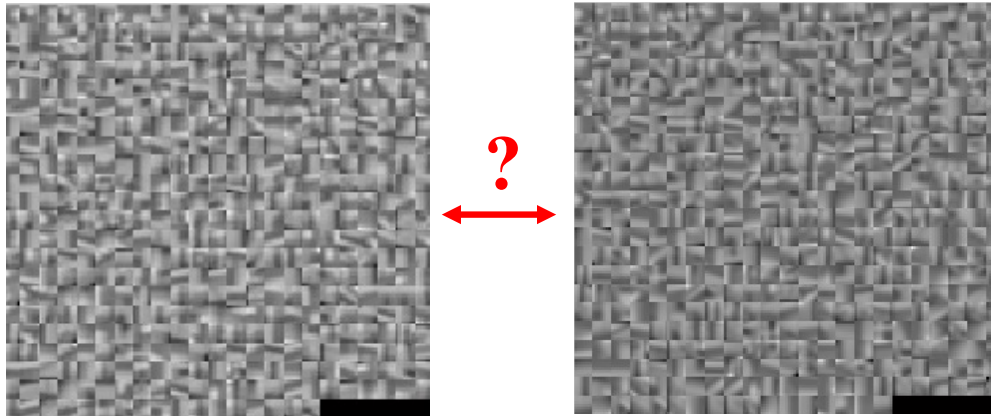
Feature descriptors: SIFT

- Descriptor computation:
 - Divide patch into 4×4 sub-patches
 - Compute histogram of gradient orientations (8 reference angles) inside each sub-patch
 - Resulting descriptor: $4 \times 4 \times 8 = 128$ dimensions
- Advantages of SIFT descriptor over raw pixel values
 - Gradients are less sensitive to illumination change
 - Pooling of gradients over the sub-patches achieves robustness to small shifts, but still preserves some spatial information

D. Lowe. [Distinctive image features from scale-invariant keypoints](#). *IJCV* 60 (2), pp. 91-110, 2004

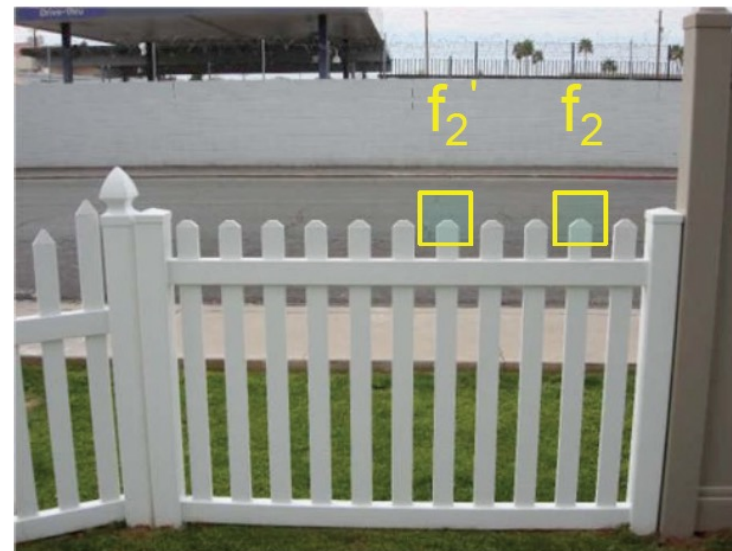
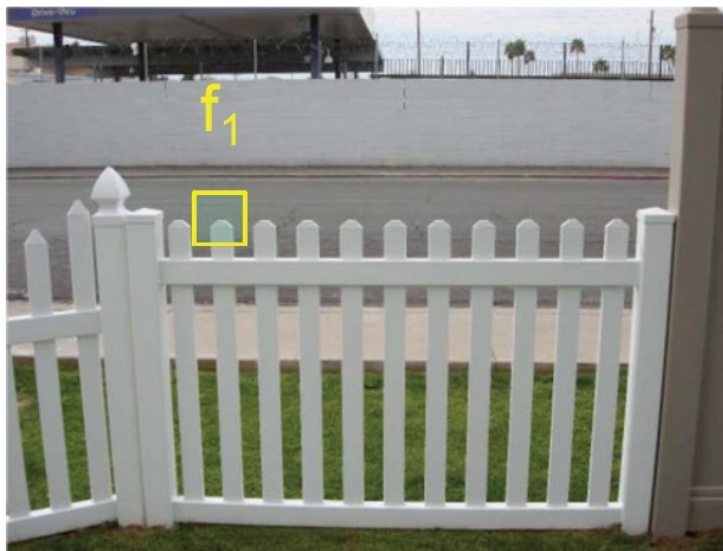
Generating putative correspondences

- For each patch in one image, find a short list of patches in the other image that could match it based solely on appearance



Rejection of ambiguous matches

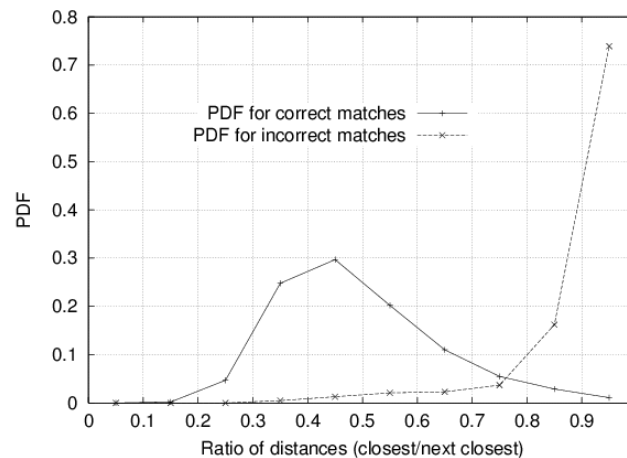
- How can we tell which putative matches are more reliable?
- Heuristic: compare distance of **nearest** neighbor to that of **second nearest** neighbor



Source: Y. Furukawa

Rejection of ambiguous matches

- How can we tell which putative matches are more reliable?
- Heuristic: compare distance of **nearest** neighbor to that of **second nearest** neighbor
 - Ratio of closest distance to second-closest distance will be **high** for features that are **not** distinctive



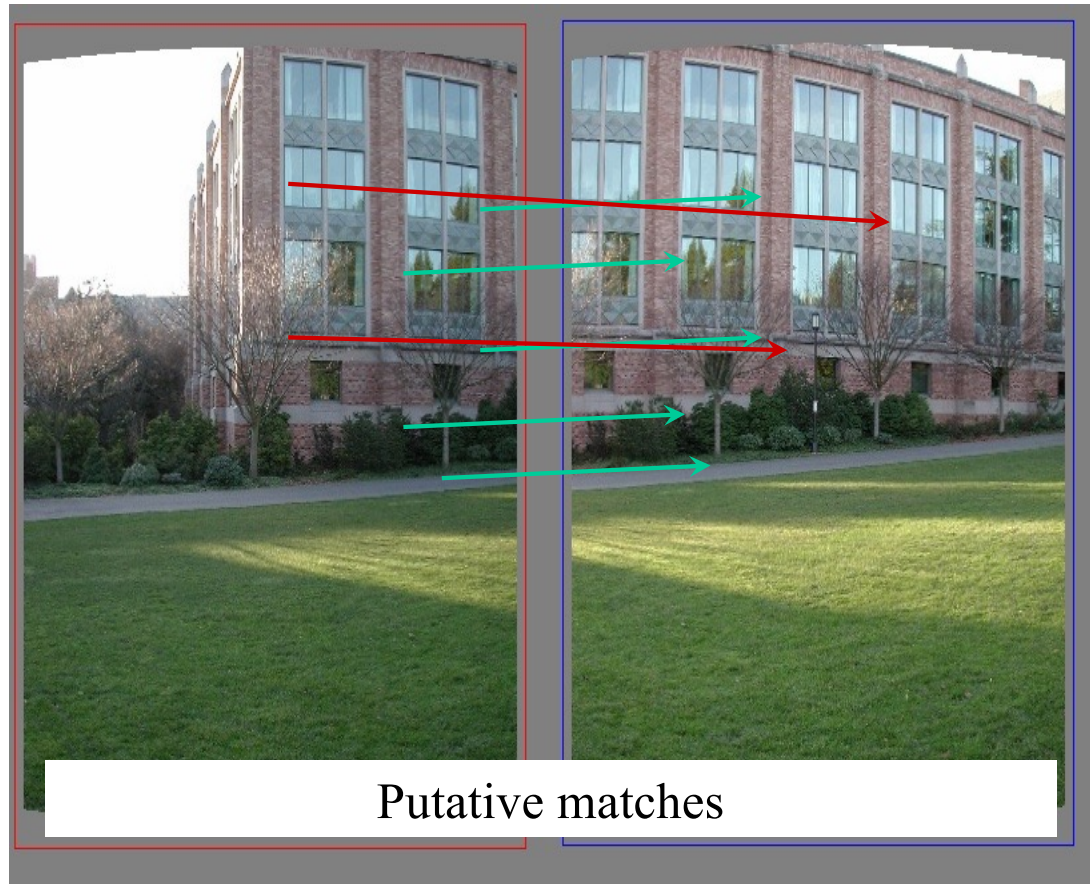
Threshold of 0.8 found to provide good separation

D. Lowe. [Distinctive image features from scale-invariant keypoints](#). *IJCV* 60 (2), pp. 91-110, 2004

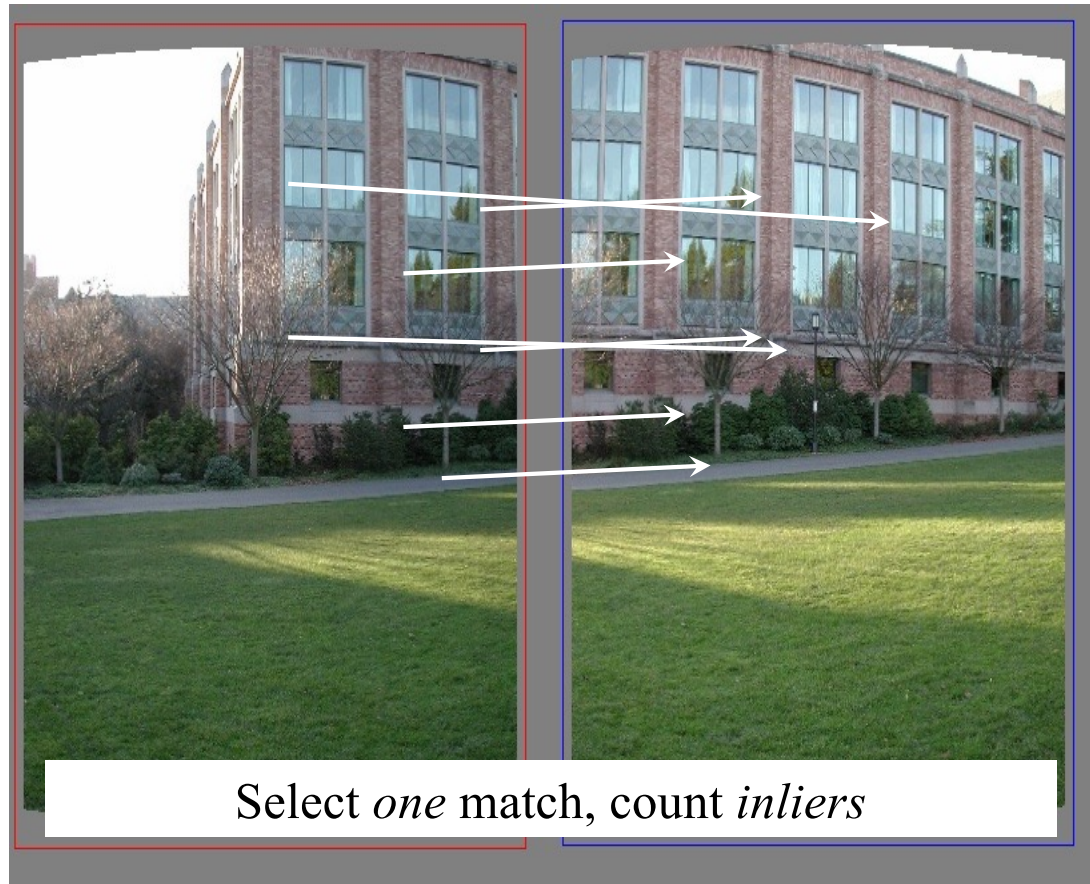
Robust alignment

- Even after filtering out ambiguous matches, the set of putative matches still contains a very high percentage of outliers
- Solution: RANSAC
- RANSAC loop:
 1. Randomly select a *seed group* of matches
 2. Compute transformation from seed group
 3. Find *inliers* to this transformation
 4. If the number of inliers is sufficiently large, re-compute least-squares estimate of transformation on all of the inliers
- At the end, keep the transformation with the largest number of inliers

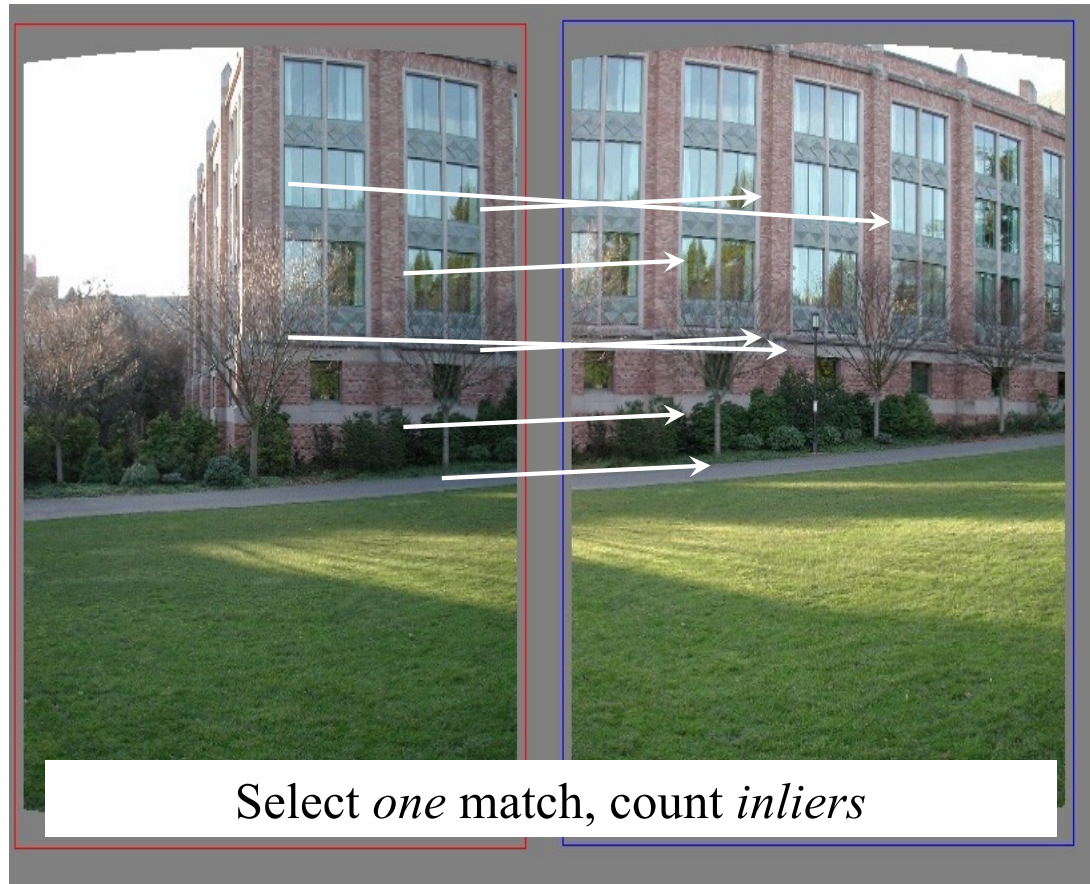
RANSAC example: Translation



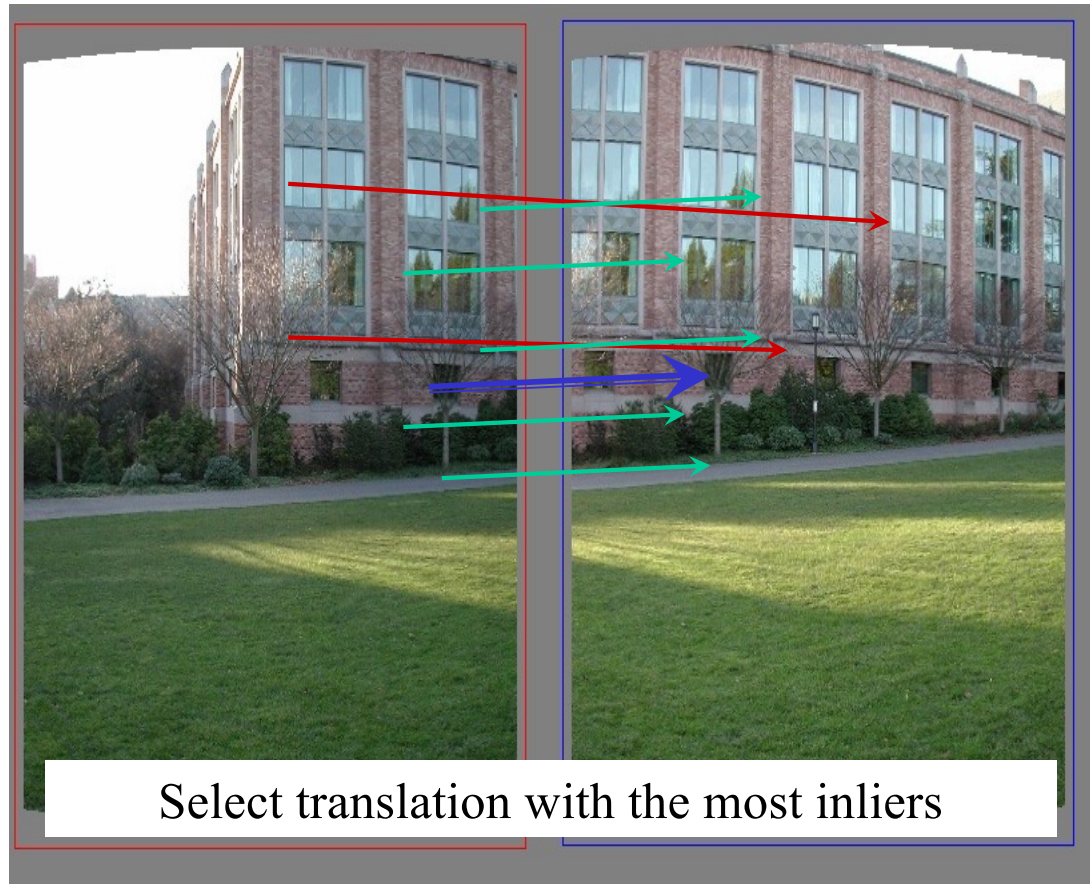
RANSAC example: Translation



RANSAC example: Translation

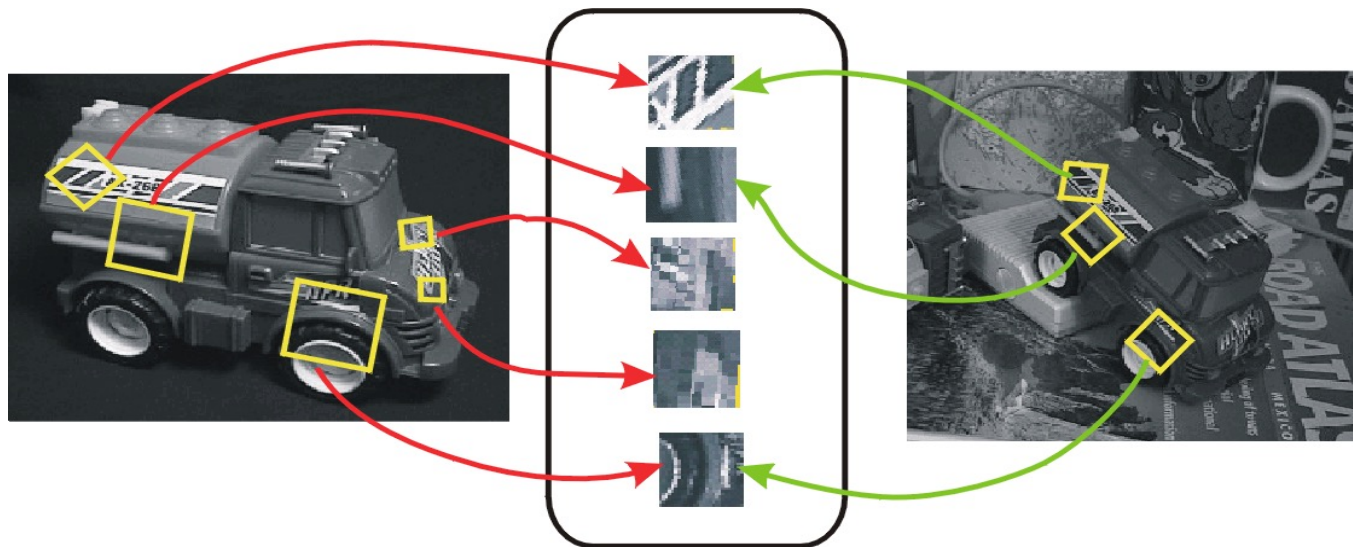


RANSAC example: Translation



Alternative to RANSAC: Voting

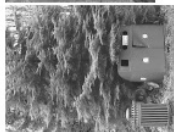
- A single SIFT match can vote for translation, rotation, and scale parameters of a transformation between two images
 - Votes are accumulated in a 4D coarsely discretized parameter space
 - Clusters of matches falling into the same bin undergo a more precise verification procedure



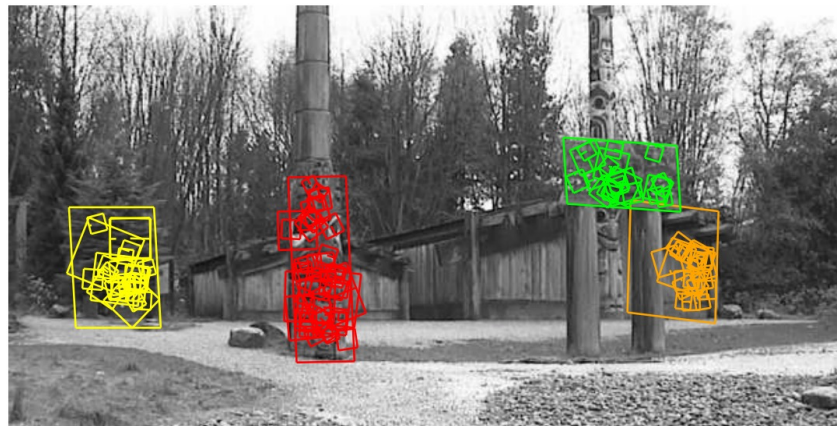
D. Lowe. [Distinctive image features from scale-invariant keypoints](#). *IJCV* 60 (2), pp. 91-110, 2004

Lowe paper results

Model images



Test image



D. Lowe. [Distinctive image features from scale-invariant keypoints](#). *IJCV* 60 (2), pp. 91-110, 2004

Alignment: Overview

- Motivation
- Fitting of transformations
 - Affine transformations
 - Homographies
- Robust alignment
 - Descriptor-based feature matching
 - RANSAC
- Learning-based feature extraction and matching

Learning-based feature extraction

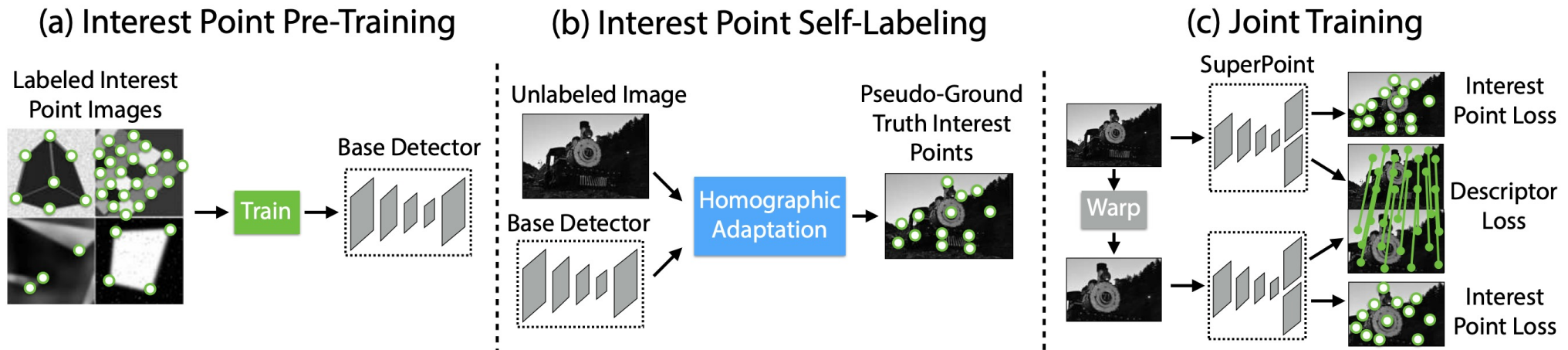


Figure 2. **Self-Supervised Training Overview.** In our self-supervised approach, we (a) pre-train an initial interest point detector on synthetic data and (b) apply a novel Homographic Adaptation procedure to automatically label images from a target, unlabeled domain. The generated labels are used to (c) train a fully-convolutional network that jointly extracts interest points and descriptors from an image.

Learning-based feature extraction

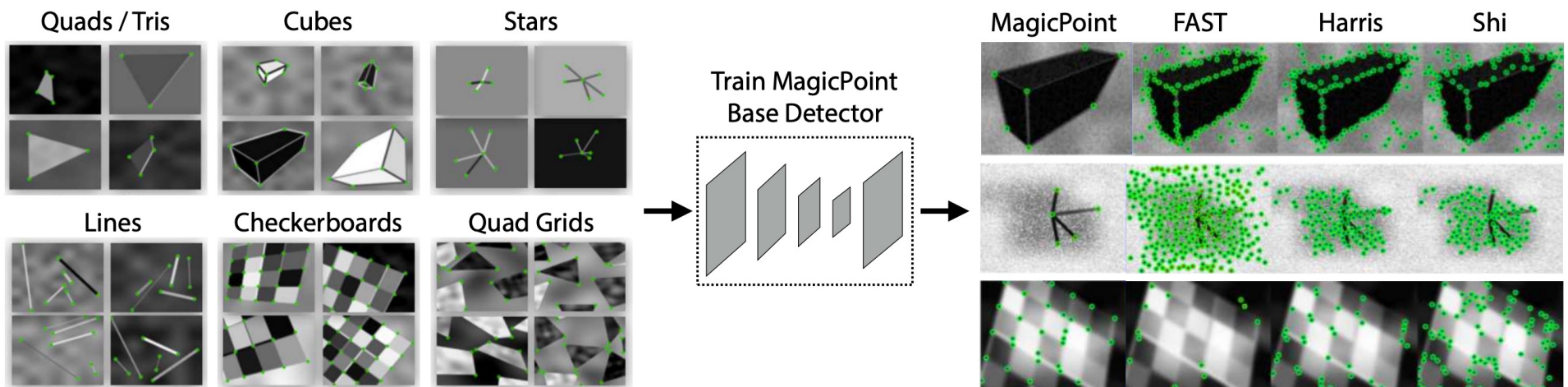


Figure 4. **Synthetic Pre-Training.** We use our Synthetic Shapes dataset consisting of rendered triangles, quadrilaterals, lines, cubes, checkerboards, and stars each with ground truth corner locations. The dataset is used to train the MagicPoint convolutional neural network, which is more robust to noise when compared to classical detectors.

Learning-based feature extraction

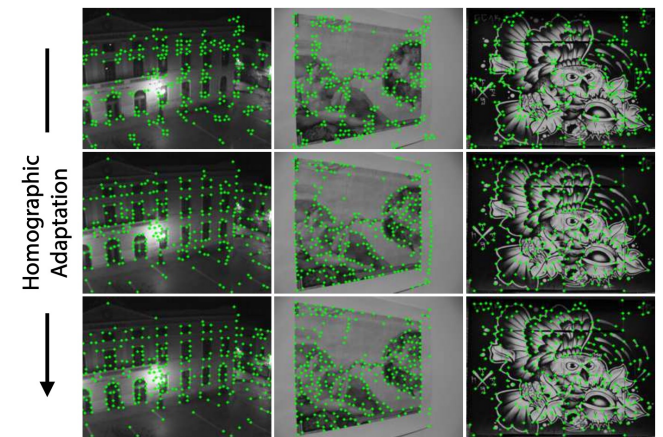
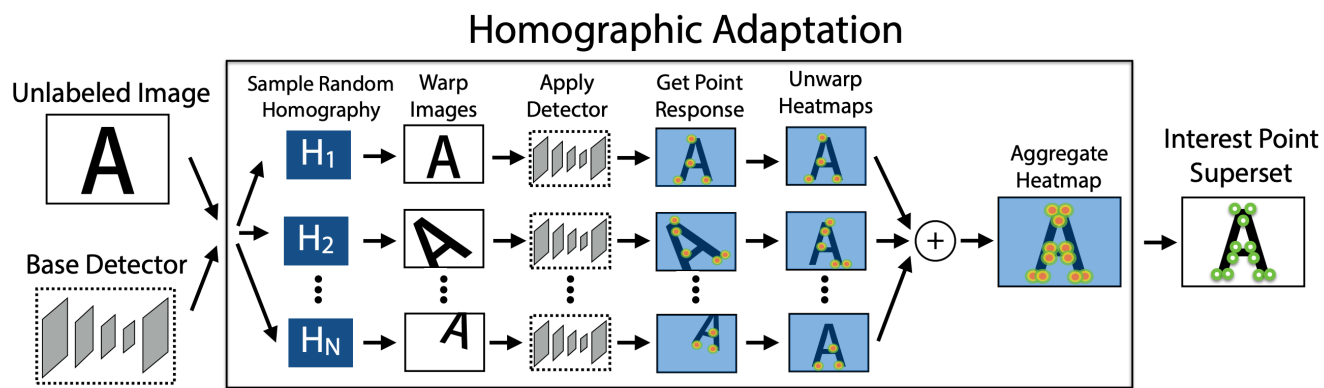


Figure 7. **Iterative Homographic Adaptation.** Top row: initial base detector (MagicPoint) struggles to find repeatable detections. Middle and bottom rows: further training with Homographic Adaptation improves detector performance.

Learning-based image matching

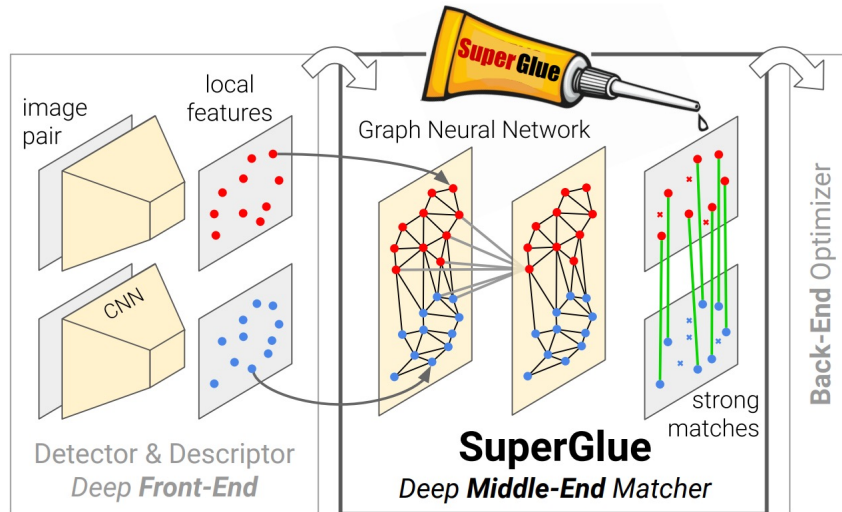


Figure 1: **Feature matching with SuperGlue.** Our approach establishes pointwise correspondences from off-the-shelf local features: it acts as a middle-end between hand-crafted or learned front-end and back-end. SuperGlue uses a graph neural network and attention to solve an assignment optimization problem, and handles partial point visibility and occlusion elegantly, producing a partial assignment.

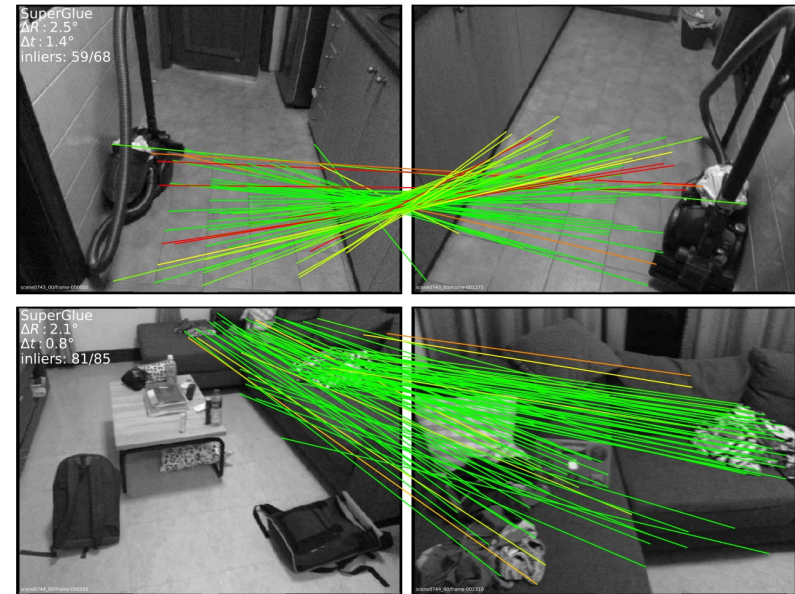
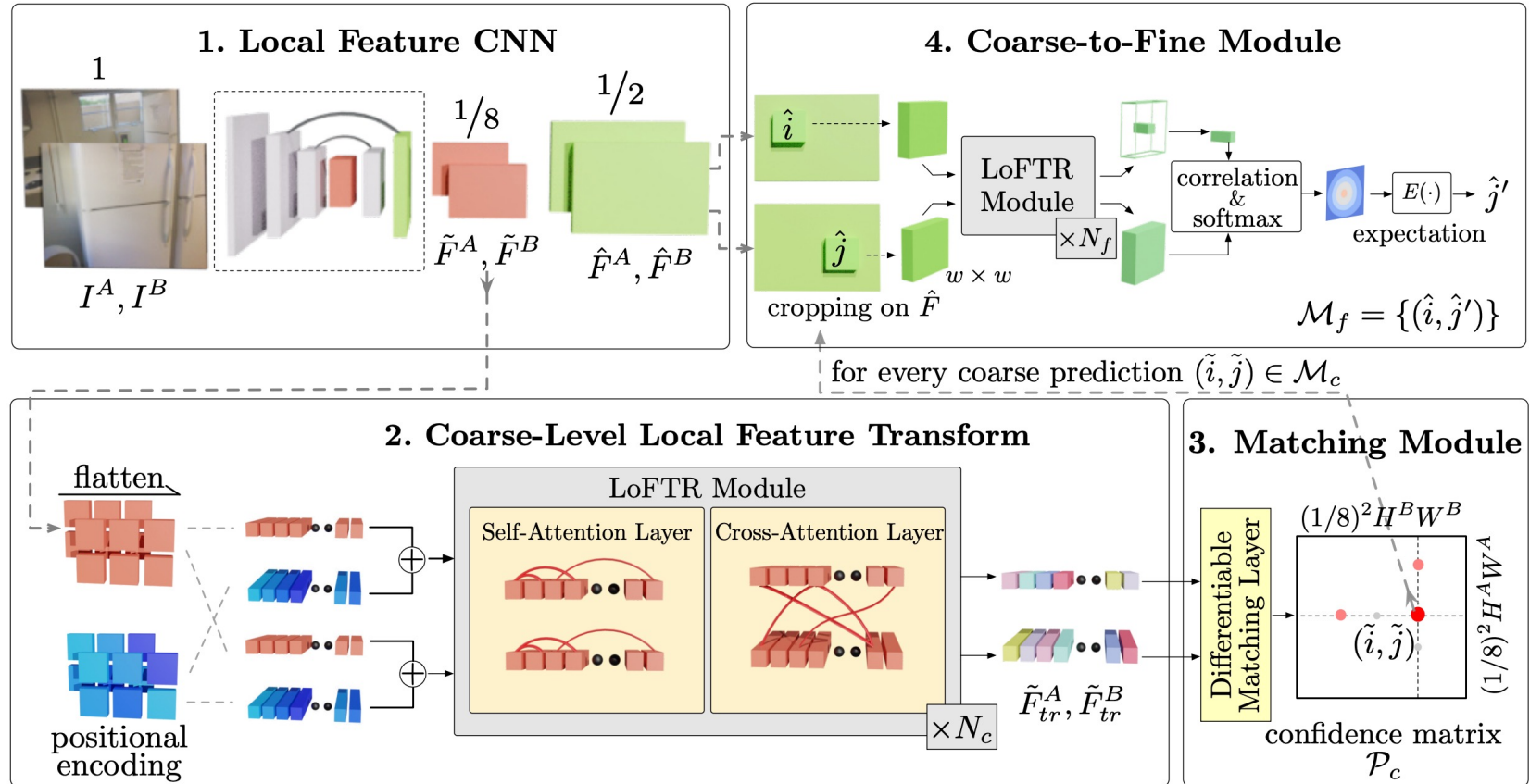


Figure 2: **SuperGlue correspondences.** For these two challenging indoor image pairs, matching with SuperGlue results in accurate poses while other learned or handcrafted methods fail (correspondences colored by epipolar error).

Learning-based image matching



Learning-based image matching

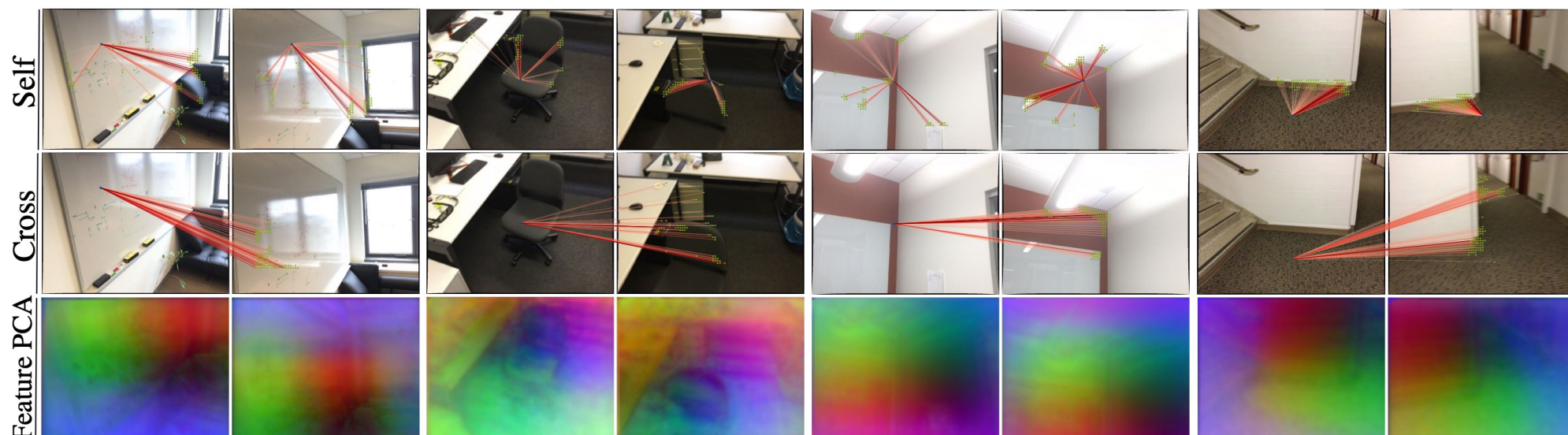
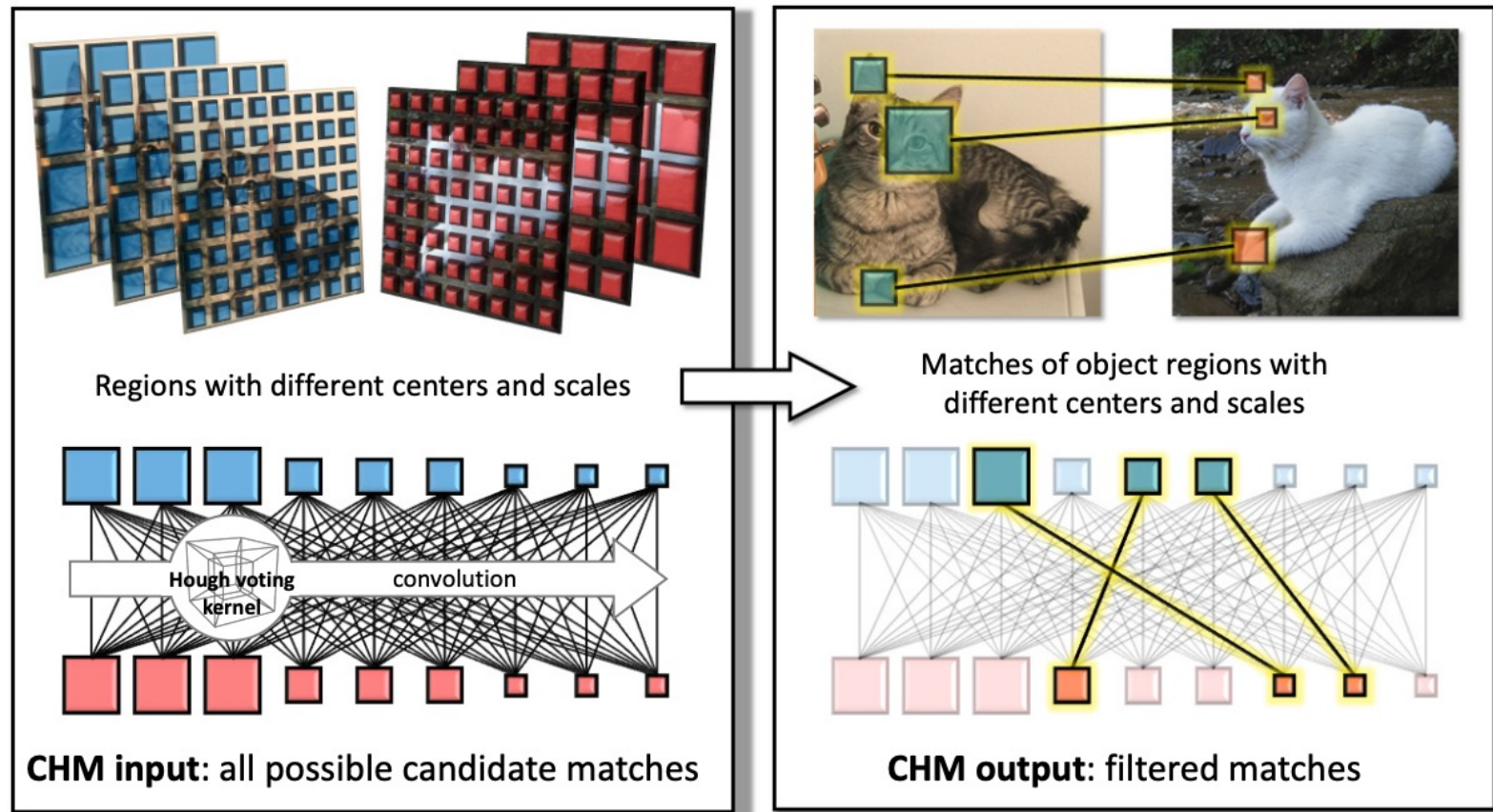


Figure 6: **Visualization of self and cross attention weights and the transformed features.** In the first two examples, the query point from the low-texture region is able to aggregate the surrounding global information flexibly. For instance, the point on the chair is looking at the edge of the chair. In the last two examples, the query point from the distinctive region can also utilize the richer information from other regions. The feature visualization with PCA further shows that LoFTR learns a position-dependent feature representation.

Learning-based image matching

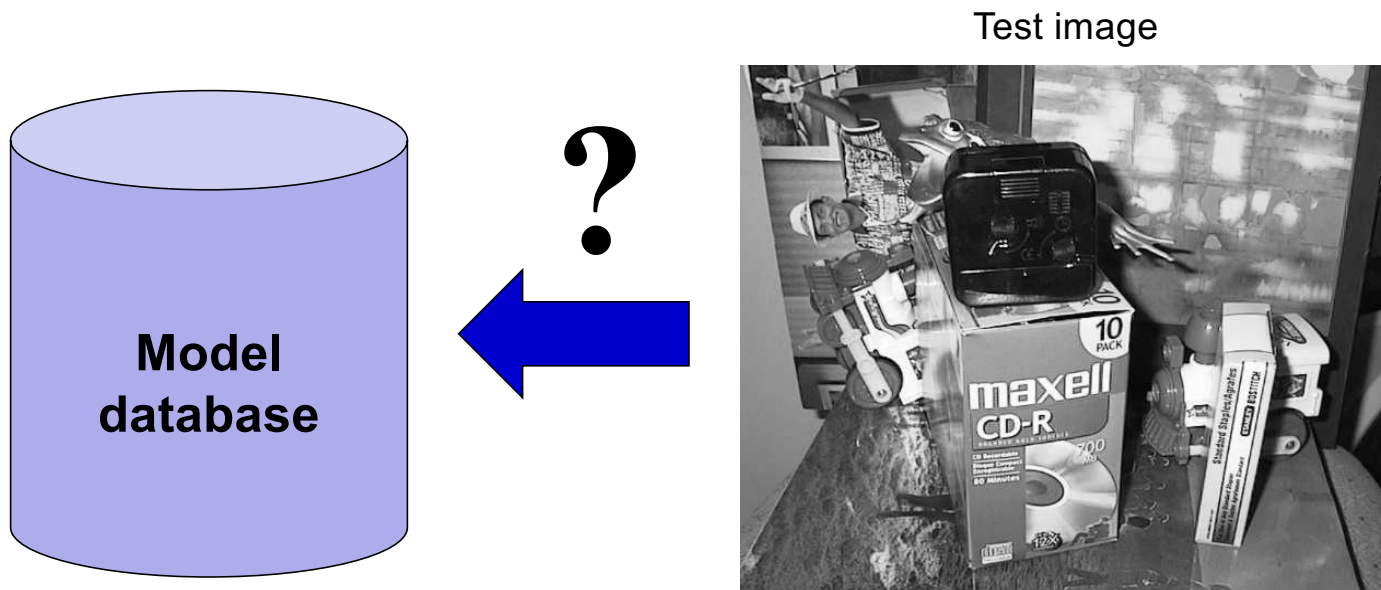


Alignment: Overview

- Motivation
- Fitting of transformations
 - Affine transformations
 - Homographies
- Robust alignment
 - Descriptor-based feature matching
 - RANSAC
- Learning-based feature extraction and matching
- Large-scale alignment

Scalability: Alignment to large databases

- What if we need to align a test image with thousands or millions of images in a model database?
 - Efficient putative match generation: approximate descriptor similarity search, inverted indices



Large-scale visual search

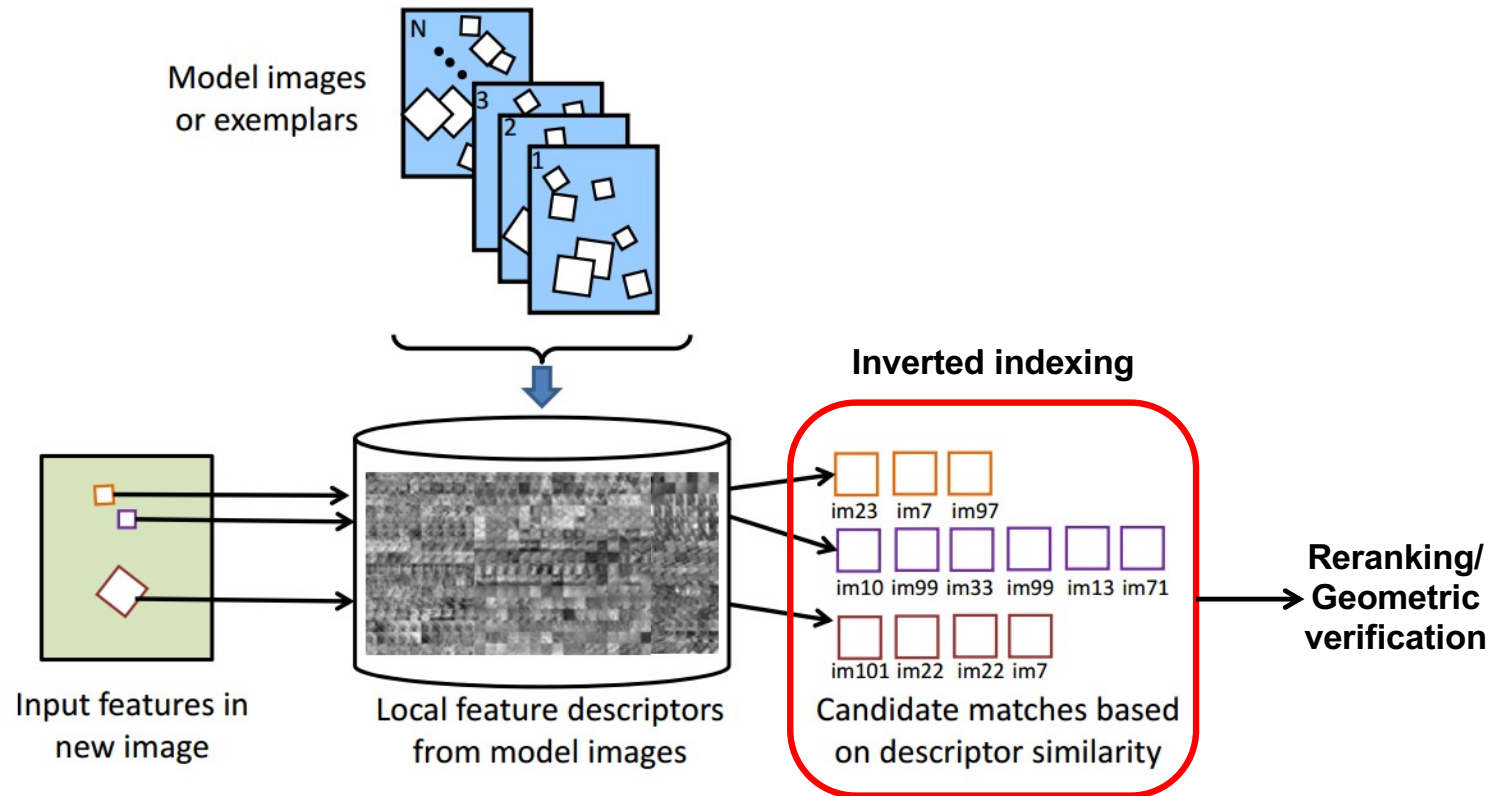
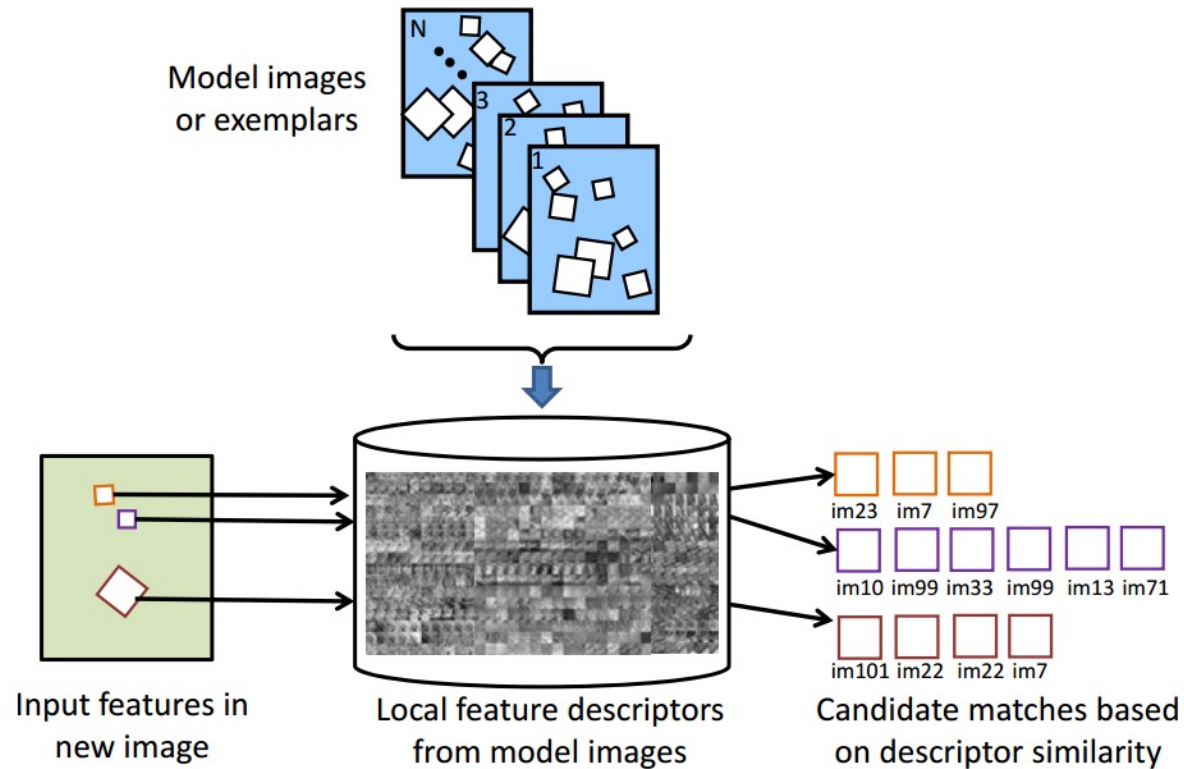


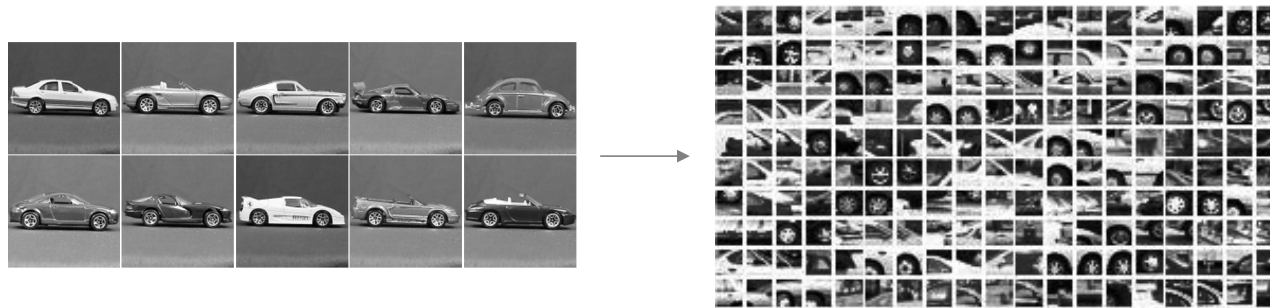
Figure from: Kristen Grauman and Bastian Leibe, [Visual Object Recognition](#), Synthesis Lectures on Artificial Intelligence and Machine Learning, April 2011, Vol. 5, No. 2, pp. 1-181

How to do the indexing?

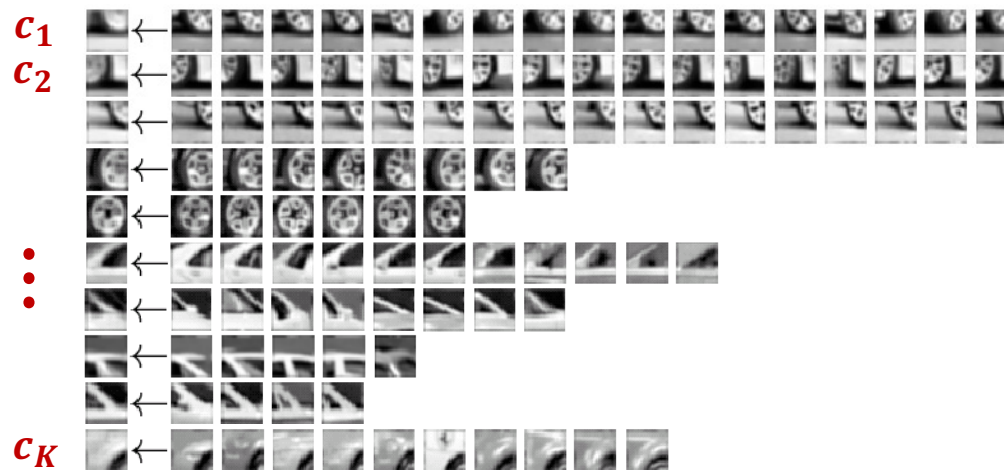


- Idea: find a set of *visual codewords* to which descriptors can be *quantized*

Recall: Visual codebook for implicit shape models



Appearance codebook



B. Leibe, A. Leonardis, and B. Schiele, [Combined Object Categorization and Segmentation with an Implicit Shape Model](#), ECCV Workshop on Statistical Learning in Computer Vision 2004

K-means clustering

- We want to find K cluster centers and an assignment of points to cluster centers to minimize the sum of squared Euclidean distances between each point and its assigned cluster center:

$$\sum_i \sum_k a_{ik} \|x_i - c_k\|^2$$

Sum over all points

Sum over all clusters

Point x_i is assigned to cluster k

Center of cluster k

K-means clustering

- We want to find K cluster centers and an assignment of points to cluster centers to minimize the sum of squared Euclidean distances between each point and its assigned cluster center:

$$\sum_i \sum_k a_{ik} \|x_i - c_k\|^2$$

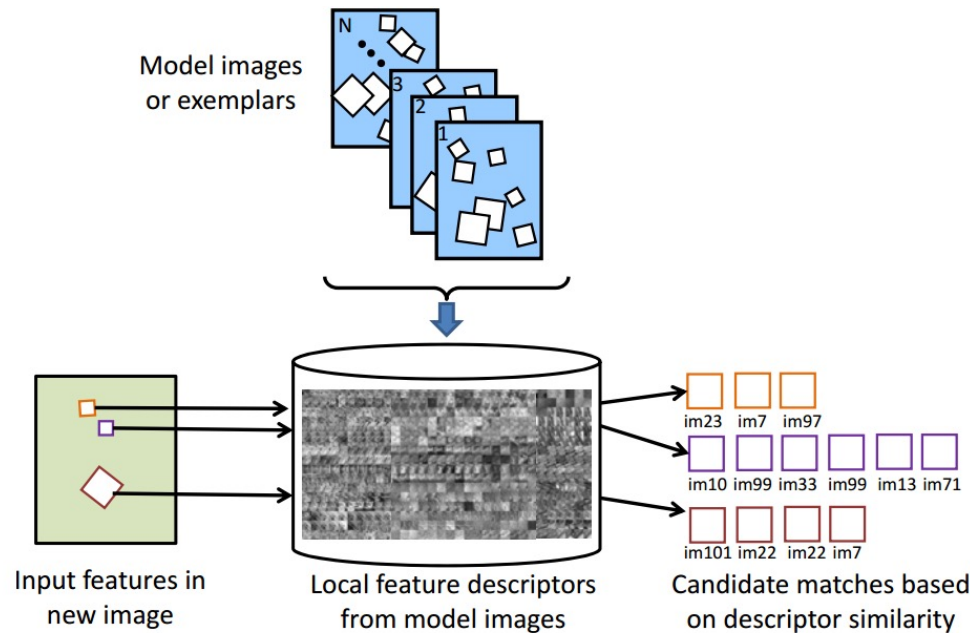
- Algorithm:
 - Randomly initialize K cluster centers
 - Iterate until convergence:
 - Assign each data point to its nearest center
 - Recompute each cluster center as the mean of all points assigned to it

K-means example



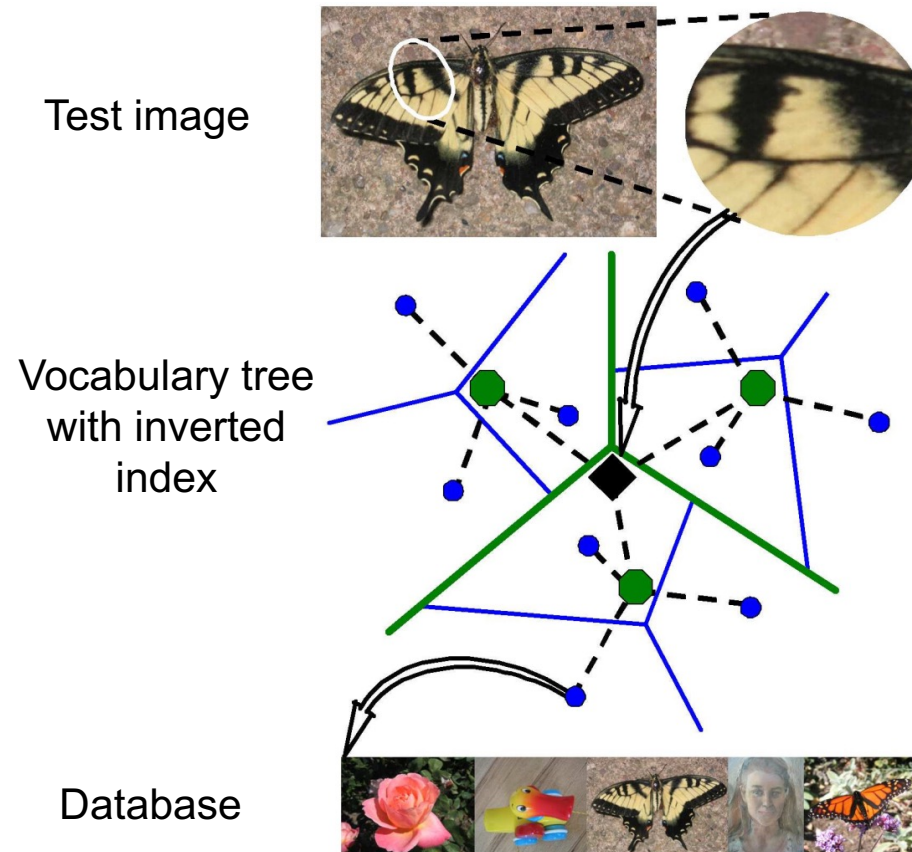
[Source](#)

How to do the indexing?

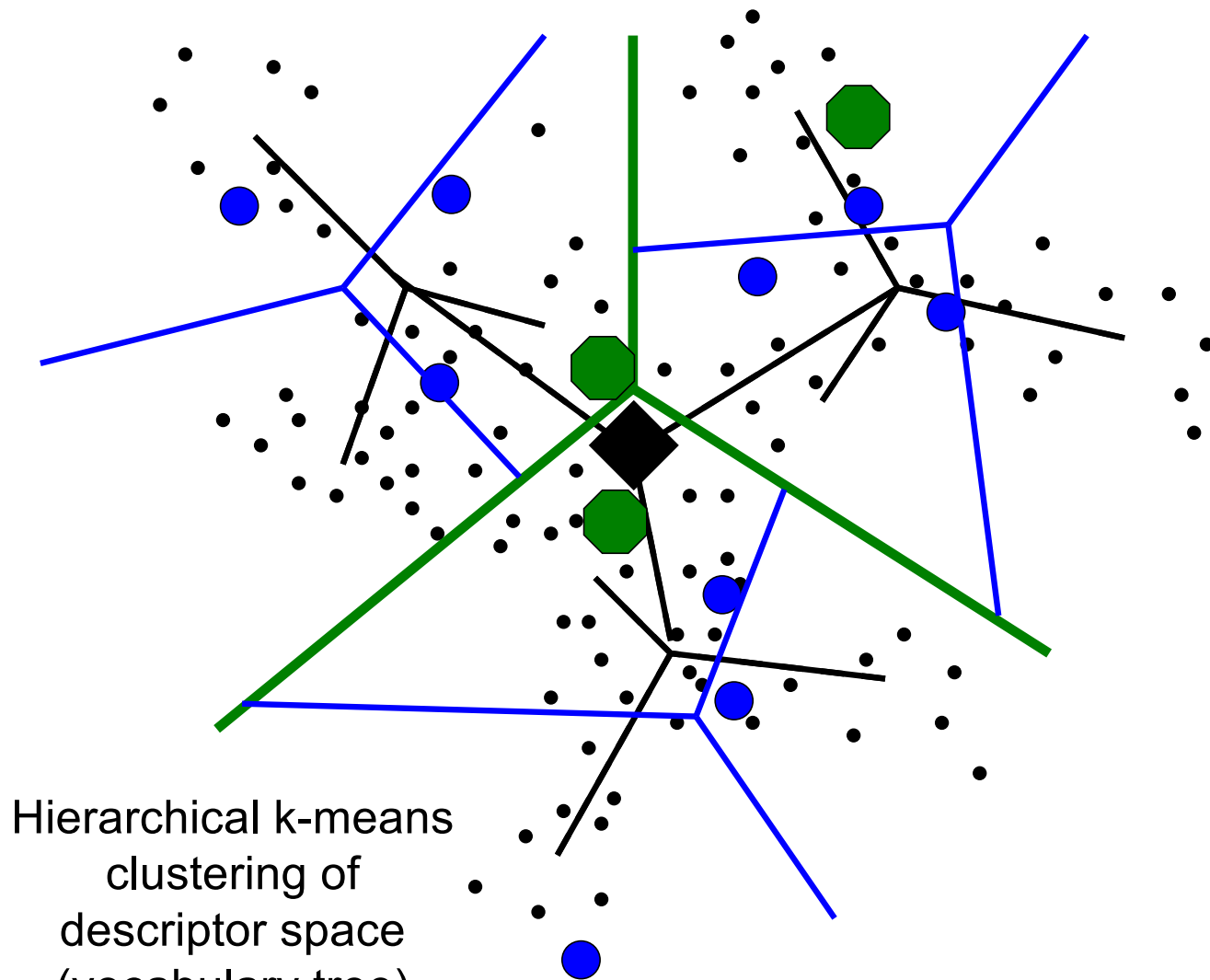


- Cluster descriptors in the database to form codebook
- At query time, quantize descriptors in query image to nearest codevectors
- Problem solved?

Efficient indexing technique: Vocabulary trees

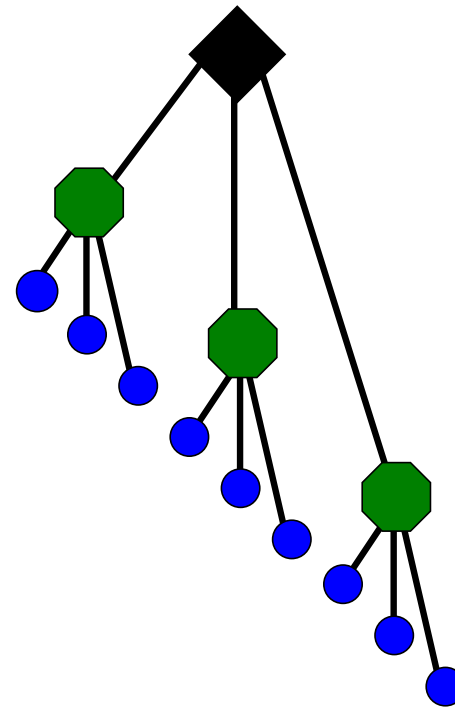


D. Nistér and H. Stewénus, [Scalable Recognition with a Vocabulary Tree](#), CVPR 2006



Hierarchical k-means
clustering of
descriptor space
(vocabulary tree)

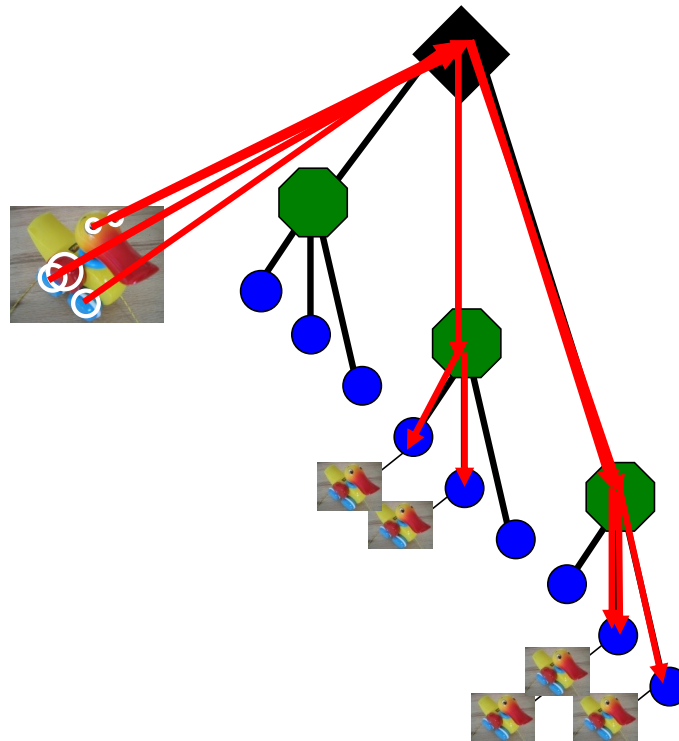
Slide credit: D. Nister



Vocabulary tree/inverted index

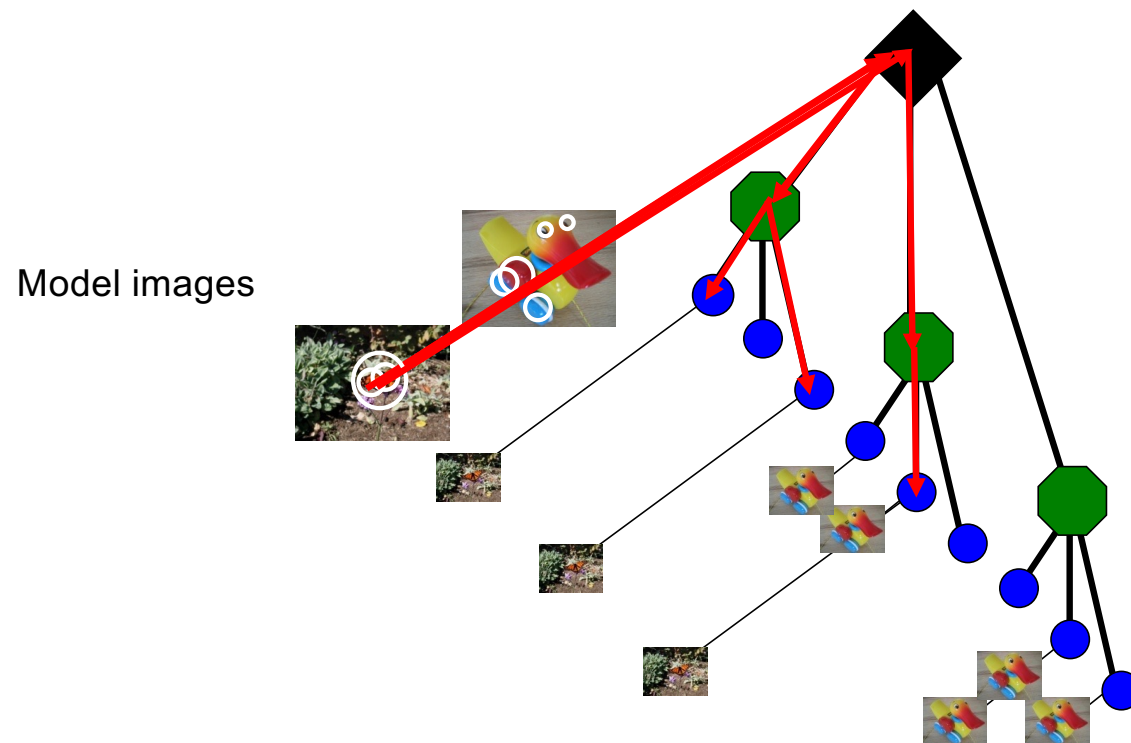
Slide credit: D. Nister

Model images



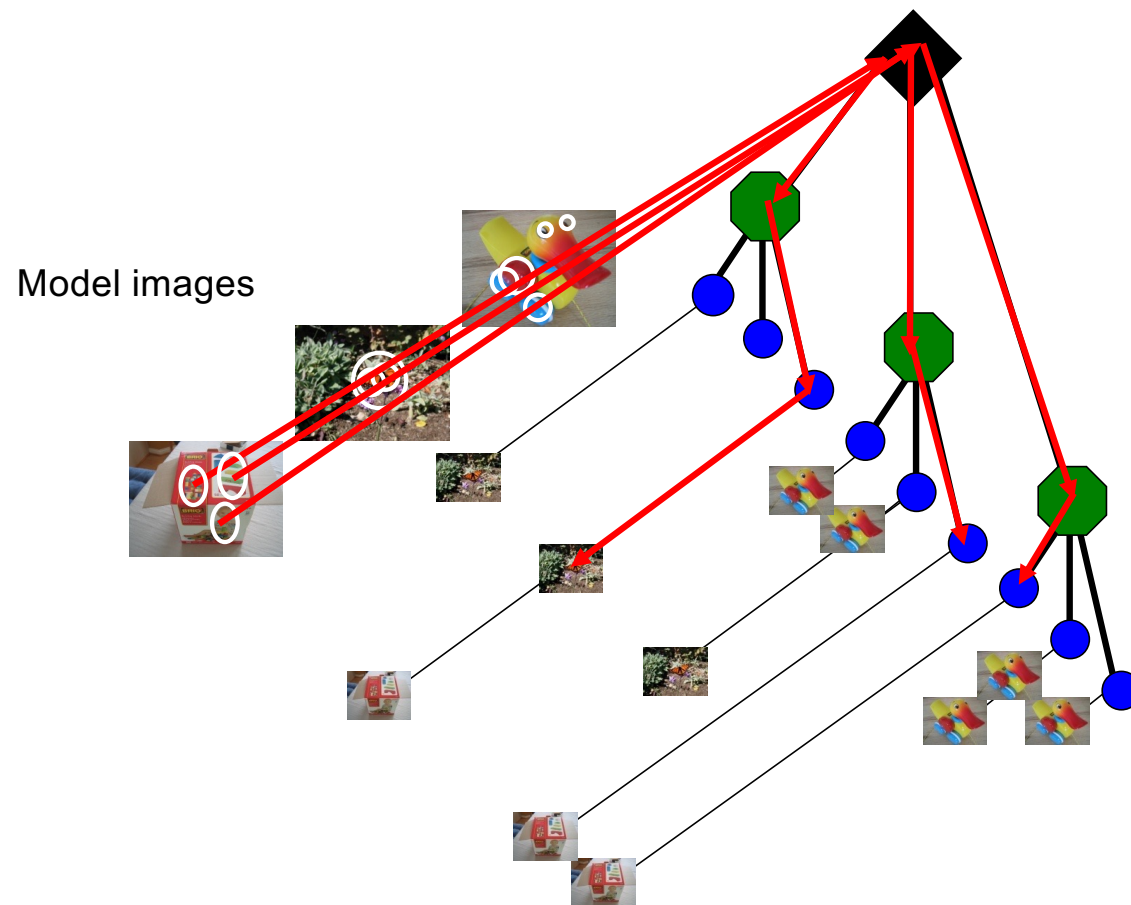
Populating the vocabulary tree/inverted index

Slide credit: D. Nister



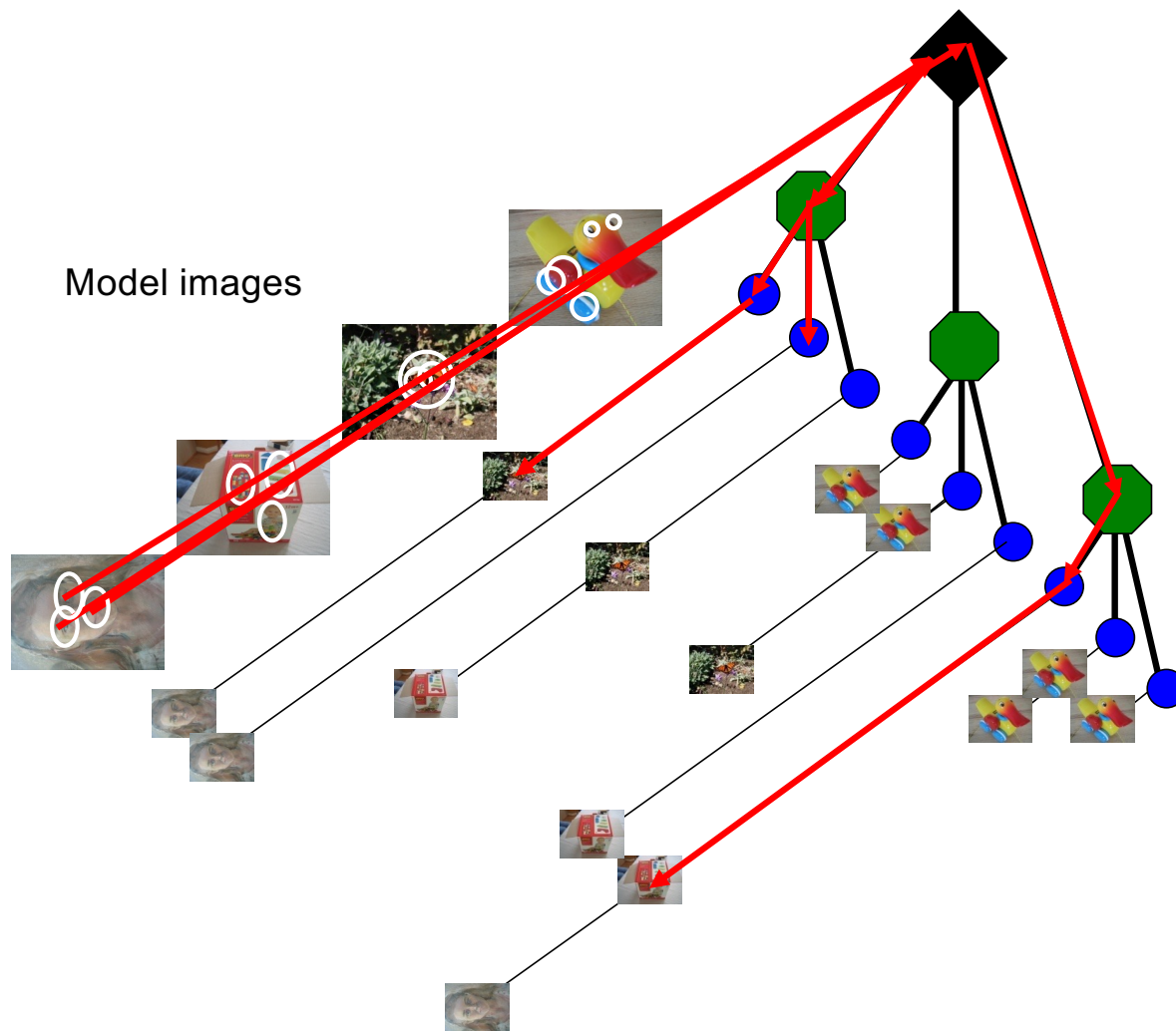
Populating the vocabulary tree/inverted index

Slide credit: D. Nister



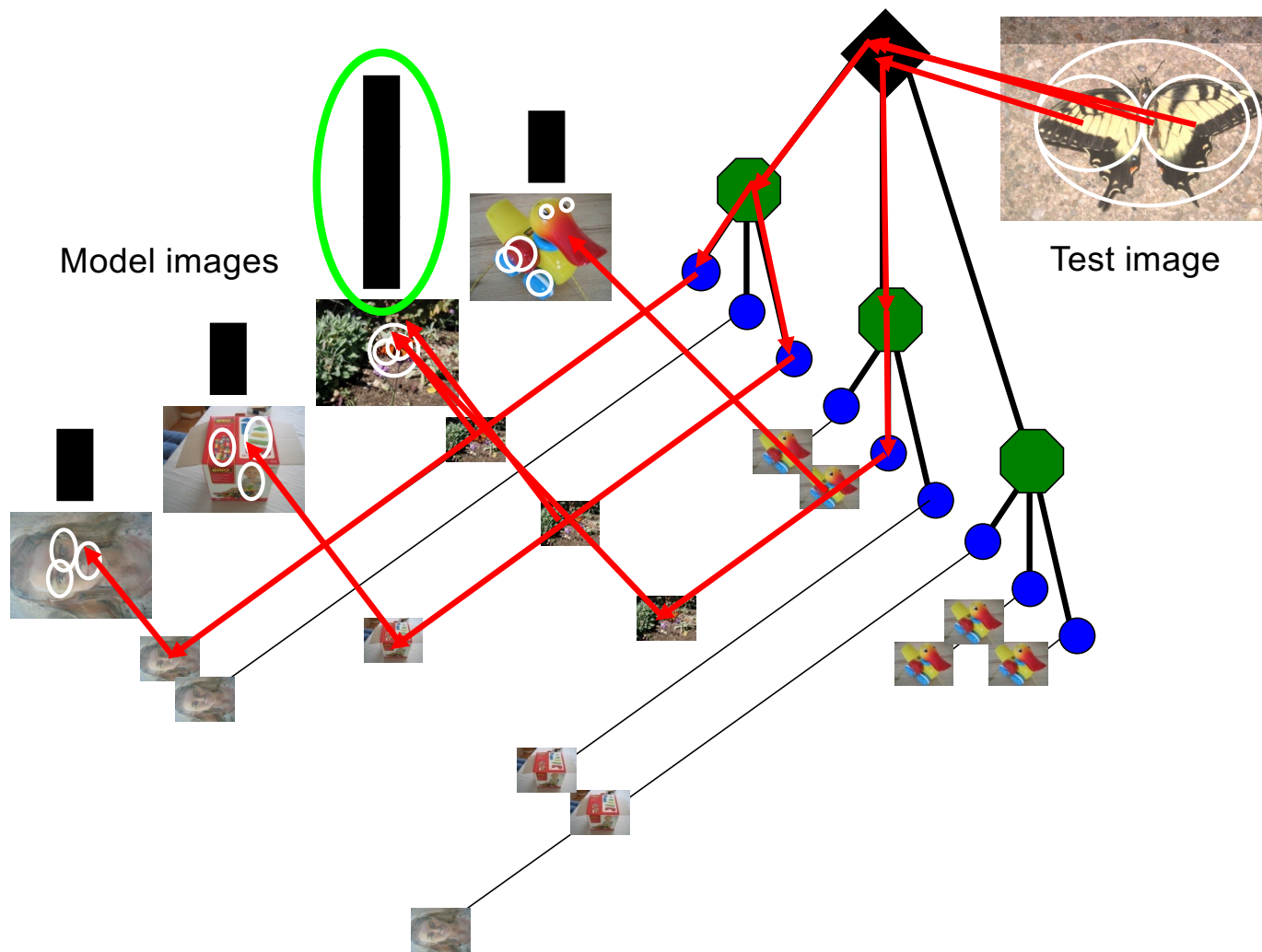
Populating the vocabulary tree/inverted index

Slide credit: D. Nister



Populating the vocabulary tree/inverted index

Slide credit: D. Nister



Looking up a test image

Slide credit: D. Nister